

Java

Violet

2026-04-14

Chapter 0

目 录

II	JVM 标准库与常用工具
1.	字符串处理 ····· 7
1.1.	String 类详解 ····· 7
1.2.	StringBuilder 与 StringBuffer ····· 10
1.3.	正则表达式 ····· 12
1.4.	编码与字符集 ····· 16
1.5.	高级字符串处理 ····· 17
1.6.	性能最佳实践 ····· 18
1.7.	常见陷阱 ····· 19
1.8.	总结 ····· 20
2.	集合框架 ····· 21
2.1.	集合类概述 ····· 21
2.2.	List 接口与实现 ····· 21
2.3.	Set 接口与实现 ····· 23
2.4.	Map 接口与实现 ····· 24
2.5.	ConcurrentHashMap 原理 ····· 26
2.6.	集合遍历与 fail-fast ····· 26
2.7.	多线程集合类 ····· 27
2.8.	深拷贝 ····· 27
2.9.	补充说明 ····· 28
2.10.	总结 ····· 30
3.	异常处理 ····· 31
3.1.	异常概述 ····· 31
3.2.	异常类型与层次结构 ····· 32
3.3.	异常处理机制 ····· 33
3.4.	自定义异常 ····· 36
3.5.	最佳实践与反模式 ····· 36
3.6.	面试与实战高频 ····· 38
IV	并发编程

4. 并发基础与问题模型 41

4.1. 进程与线程、并发 vs 并行 41

4.2. Java 线程生命周期与操作 42

4.3. 并发核心问题：三性模型 44

5. 线程安全与同步机制 46

5.1. synchronized 与 Lock 46

5.2. CAS 与原子类 48

5.3. 线程安全集合与并发容器 50

6. 并发协作与线程池 52

6.1. 线程通信 52

6.2. 线程池 57

6.3. JUC 工具类 61

6.4. 总结 65

7. Kotlin 协程基础 67

7.1. 协程模型 67

7.2. 核心概念 69

7.3. 协程上下文与调度器 75

7.4. 实战示例 78

7.5. 总结 80

8. 协程进阶与异步流 81

9. 并发诊断与综合实战 82

VI

JVM 底层原理与性能调优

10. GraalVM 85

10.1. GraalVM 概述 85

10.2. Native Image 详解 86

10.3. 配置与优化 89

10.4. Spring Boot 与 GraalVM 92

10.5. 多语言编程 95

10.6. 性能调优 97

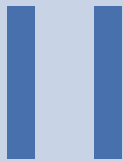
10.7. 实际应用场景 98

10.8. 局限性与注意事项 100

10.9. 生态系统与支持 102



Java & Kotlin 核心基础



JVM 标准库与常用工具

1.	字符串处理	7
2.	集合框架	21
3.	异常处理	31

Chapter 1

字符串处理

字符串是 Java 中最常用的数据类型之一。Java 提供了丰富的 API 来处理字符串，包括不可变的 `String`、可变的 `StringBuilder` 和 `StringBuffer`，以及强大的正则表达式支持。

NOTE

Java 字符串的核心特性是**不可变性**（Immutability）。一旦创建，String 对象的内容就不能被修改。这个设计带来了线程安全、缓存友好等优势，但也需要注意性能问题。

1 String 类详解

1.1 字符串的不可变性

String 对象一旦创建，其内容就不能改变：

```
1 String s = "Hello";
2 s.concat(" World"); // 返回新字符串，s本身不变
3 System.out.println(s); // 输出：Hello
4
5 String t = s.concat(" World"); // 需要接收返回值
6 System.out.println(t); // 输出：Hello World
```

 Java

为什么设计为不可变：

1. 安全性：防止恶意代码修改字符串
2. 线程安全：多个线程可以安全共享
3. 缓存哈希码：提高 HashMap 等集合的性能
4. 字符串常量池：节省内存

CAUTION

频繁拼接字符串时，不要使用 String 的 `+` 或 `concat()`，应该使用 `StringBuilder`。

1.2 字符串常量池

JVM 维护一个字符串常量池，用于存储字符串字面量：

```
1 String s1 = "Hello"; // 从常量池获取
2 String s2 = "Hello"; // 从常量池获取同一个对象
3 String s3 = new String("Hello"); // 在堆上创建新对象
4
5 System.out.println(s1 == s2); // true (同一引用)
```

 Java

```
6 System.out.println(s1 == s3);    // false (不同对象)
7 System.out.println(s1.equals(s3)); // true (内容相同)
```

intern()方法:

```
1 String s4 = new String("World").intern();
2 String s5 = "World";
3 System.out.println(s4 == s5); // true (都指向常量池)
```

 Java

TIP

使用 `equals()` 比较字符串内容, 使用 `==` 比较引用。大多数情况下应该使用 `equals()`。

1.3 常用方法

| 基本信息

```
1 String str = "Hello, World!";
2
3 // 长度
4 int len = str.length(); // 13
5
6 // 是否为空
7 boolean empty = str.isEmpty(); // false
8 boolean blank = str.isBlank(); // false (Java 11+, 忽略空白字符)
9
10 // 字符访问
11 char c = str.charAt(0); // 'H'
```

 Java

| 查找与判断

```
1 String str = "Hello, World!";
2
3 // 包含
4 boolean contains = str.contains("World"); // true
5
6 // 前缀/后缀
7 boolean startsWith = str.startsWith("Hello"); // true
8 boolean endsWith = str.endsWith("!"); // true
9
10 // 位置
11 int index = str.indexOf("World"); // 7
12 int lastIndex = str.lastIndexOf("o"); // 8
13
14 // 判断
15 boolean isDigit = Character.isDigit('5'); // true
16 boolean isLetter = Character.isLetter('a'); // true
```

 Java

| 截取与分割

```
1 String str = "Hello, World!";
2
3 // 截取子串
```

 Java

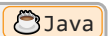

```
4 String sub1 = str.substring(7);    // "World!"
5 String sub2 = str.substring(0, 5); // "Hello"
6
7 // 分割
8 String[] parts = "apple,banana,cherry".split(",");
9 // ["apple", "banana", "cherry"]
10
11 // 限制分割次数
12 String[] limited = "a,b,c,d".split(",", 2);
13 // ["a", "b,c,d"]
```

NOTE

`split()` 的参数是正则表达式，特殊字符需要转义，如 `split("\\.")` 分割点号。

替换

```
1 String str = "Hello, World!";
2
3 // 替换所有
4 String replaced = str.replace("l", "L");
5 // "HeLLo, WorLd!"
6
7 // 替换第一个
8 String firstReplaced = str.replaceFirst("l", "L");
9 // "HeLllo, World!"
10
11 // 正则替换
12 String regexReplaced = str.replaceAll("[aeiou]", "*");
13 // "H*ll*, W*rld!"
```



大小写转换

```
1 String str = "Hello, World!";
2
3 String upper = str.toUpperCase(); // "HELLO, WORLD!"
4 String lower = str.toLowerCase(); // "hello, world!"
5
6 // 区域敏感的大小写转换
7 String turkish = "TITLE".toLowerCase(Locale.forLanguageTag("tr"));
8 // 土耳其语的'I'小写是'i'（无点）
```



修剪空白

```
1 String str = " Hello, World! ";
2
3 String trimmed = str.trim(); // "Hello, World!" (去除首尾空白)
4 String stripped = str.strip(); // "Hello, World!" (Java 11+, Unicode感知)
5 String leftStripped = str.stripLeading(); // "Hello, World! "
6 String rightStripped = str.stripTrailing(); // " Hello, World!"
```



TIP

推荐使用 `strip()` 系列方法，它们能正确处理 Unicode 空白字符，而 `trim()` 只能处理 ASCII 空白。

连接与格式化

```
1 // 连接
2 String joined = String.join(", ", "apple", "banana", "cherry");
3 // "apple, banana, cherry"
4
5 // 格式化
6 String formatted = String.format("%s is %d years old", "Alice", 30);
7 // "Alice is 30 years old"
8
9 // 文本块 (Java 15+)
10 String html = """
11     <html>
12         <body>
13             <p>Hello, World!</p>
14         </body>
15     </html>
16     """;
```

1.4 String vs StringBuilder vs StringBuffer

特性	String	StringBuilder	StringBuffer
可变性	不可变	可变	可变
线程安全	是	否	是
性能	低 (频繁修改)	高	中
适用场景	不常修改	单线程频繁修改	多线程频繁修改
同步	N/A	不同步	同步

2 StringBuilder 与 StringBuffer

2.1 StringBuilder (推荐)

StringBuilder 是可变的字符序列，适合频繁修改字符串的场景。

基本用法

```
1 StringBuilder sb = new StringBuilder();
2
3 // 追加
4 sb.append("Hello");
5 sb.append(", ");
6 sb.append("World");
7 sb.append("!");
8
```

```
9 String result = sb.toString(); // "Hello, World!"
```

链式调用

```
1 StringBuilder sb = new StringBuilder()
2   .append("Hello")
3   .append(", ")
4   .append("World")
5   .append("!");
6
7 System.out.println(sb.toString());
```

 Java

插入与删除

```
1 StringBuilder sb = new StringBuilder("Hello World");
2
3 // 插入
4 sb.insert(5, ","); // "Hello, World"
5
6 // 删除
7 sb.delete(5, 7);   // "HelloWorld"
8 sb.deleteCharAt(5); // "Helloorld"
9
10 // 替换
11 sb.replace(5, 6, ", W"); // "Hello, World"
```

 Java

反转

```
1 StringBuilder sb = new StringBuilder("Hello");
2 sb.reverse(); // "olleH"
```

 Java

容量管理

```
1 StringBuilder sb = new StringBuilder();
2 System.out.println(sb.capacity()); // 16 (默认)
3
4 sb.ensureCapacity(100); // 预分配容量
5 System.out.println(sb.capacity()); // 至少100
6
7 // trimToSize - 调整容量到实际大小
8 sb.trimToSize();
```

 Java

TIP

如果知道最终字符串的大致长度，可以在构造时指定初始容量，避免频繁扩容：

```
new StringBuilder(256)
```

2.2 StringBuffer（线程安全）

StringBuffer 与 StringBuilder API 完全相同，但所有方法都是同步的：

```
1 StringBuffer sb = new StringBuffer();
```

 Java

```
2 sb.append("Hello"); // 线程安全
```

何时使用：

- 多线程环境共享 StringBuilder
- 否则优先使用 StringBuilder（性能更好）

NOTE

在现代 Java 应用中，很少直接使用 StringBuffer。如果需要线程安全，通常使用其他并发机制。

2.3 性能对比

```
1 // 测试100000次字符串拼接
2 long start, end;
3
4 // String concatenation
5 start = System.currentTimeMillis();
6 String s = "";
7 for (int i = 0; i < 100000; i++) {
8     s += "a";
9 }
10 end = System.currentTimeMillis();
11 System.out.println("String: " + (end - start) + "ms"); // ~5000ms
12
13 // StringBuilder
14 start = System.currentTimeMillis();
15 StringBuilder sb = new StringBuilder();
16 for (int i = 0; i < 100000; i++) {
17     sb.append("a");
18 }
19 end = System.currentTimeMillis();
20 System.out.println("StringBuilder: " + (end - start) + "ms"); // ~5ms
```

性能差异：StringBuilder 比 String 快 1000 倍以上

CAUTION

在循环中拼接字符串时，永远不要使用 `+=`，必须使用 StringBuilder。

3 正则表达式

Java 通过 `java.util.regex` 包提供强大的正则表达式支持。

3.1 Pattern 和 Matcher

```
1 import java.util.regex.*;
2
3 // 编译正则表达式
4 Pattern pattern = Pattern.compile("\\d+");
5
```

```
6 // 创建匹配器
7 Matcher matcher = pattern.matcher("abc123def456");
8
9 // 查找所有匹配
10 while (matcher.find()) {
11     System.out.println(matcher.group());
12 }
13 // 输出:
14 // 123
15 // 456
```

3.2 常用方法

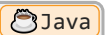
matches() - 完全匹配

```
1 boolean match = Pattern.matches("\\d+", "123"); // true
2 boolean noMatch = Pattern.matches("\\d+", "123a"); // false
```



find() - 查找子串

```
1 Pattern pattern = Pattern.compile("\\w+");
2 Matcher matcher = pattern.matcher("Hello World 123");
3
4 while (matcher.find()) {
5     System.out.println(matcher.group());
6 }
7 // 输出:
8 // Hello
9 // World
10 // 123
```



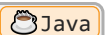
replaceAll() / replaceFirst()

```
1 String text = "The year is 2024 and month is 03";
2
3 // 替换所有数字
4 String replaced = text.replaceAll("\\d+", "*");
5 // "The year is * and month is *"
6
7 // 替换第一个
8 String firstReplaced = text.replaceFirst("\\d+", "****");
9 // "The year is **** and month is 03"
```



split() - 分割

```
1 String[] parts = "apple,banana,cherry".split(",");
2 // ["apple", "banana", "cherry"]
3
4 // 使用正则分割
5 String[] words = "one1two2three".split("\\d+");
6 // ["one", "two", "three"]
```



3.3 分组捕获

```
1 String email = "user@example.com";
2 Pattern pattern = Pattern.compile("(\\w+)@(\\w+)\\.\\.\\. (\\w+)");
3 Matcher matcher = pattern.matcher(email);
4
5 if (matcher.matches()) {
6     System.out.println("Full match: " + matcher.group(0)); // user@example.com
7     System.out.println("Username: " + matcher.group(1)); // user
8     System.out.println("Domain: " + matcher.group(2)); // example
9     System.out.println("TLD: " + matcher.group(3)); // com
10 }
```

3.4 命名分组 (Java 7+)

```
1 Pattern pattern = Pattern.compile("(?<username>\\w+)@(?(?<domain>\\w+)\\.\\. (?(?<tld>\\w+)");
2 Matcher matcher = pattern.matcher("user@example.com");
3
4 if (matcher.matches()) {
5     System.out.println(matcher.group("username")); // user
6     System.out.println(matcher.group("domain")); // example
7     System.out.println(matcher.group("tld")); // com
8 }
```

TIP

命名分组使正则表达式更易读和维护，推荐使用。

3.5 常用正则模式

验证邮箱

```
1 String emailRegex = "^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$";
2 boolean isValid = email.matches(emailRegex);
```

验证手机号（中国）

```
1 String phoneRegex = "^1[3-9]\\d{9}$";
2 boolean isValid = phone.matches(phoneRegex);
```

验证身份证号

```
1 String idRegex = "^\\d{17}[\\dXx]$";
2 boolean isValid = idCard.matches(idRegex);
```

提取 URL

```
1 String urlRegex = "https?:/[^\s]+";
2 Pattern pattern = Pattern.compile(urlRegex);
3 Matcher matcher = pattern.matcher(text);
4
```

```

5 while (matcher.find()) {
6     System.out.println(matcher.group());
7 }

```

3.6 正则表达式语法速查

元字符	含义	示例
<code>\.</code>	任意字符	<code>`a.c`</code> 匹配 <code>abc</code> , <code>a1c</code>
<code>\d</code>	数字	<code>`\d+`</code> 匹配 <code>123</code>
<code>\w</code>	单词字符	<code>`\w+`</code> 匹配 <code>hello</code>
<code>\s</code>	空白字符	<code>`\s+`</code> 匹配空格
<code>*</code>	0 次或多次	<code>`ab*c`</code> 匹配 <code>ac</code> , <code>abc</code> , <code>abbc</code>
<code>+</code>	1 次或多次	<code>`ab+c`</code> 匹配 <code>abc</code> , <code>abbc</code>
<code>?</code>	0 次或 1 次	<code>`ab?c`</code> 匹配 <code>ac</code> , <code>abc</code>
<code>{n}</code>	恰好 n 次	<code>`a{3}`</code> 匹配 <code>aaa</code>
<code>{n,m}</code>	n 到 m 次	<code>`a{2,4}`</code> 匹配 <code>aa</code> , <code>aaa</code> , <code>aaaa</code>
<code>^</code>	行首	<code>`^Hello`</code> 匹配行首的 <code>Hello</code>
<code>\$</code>	行尾	<code>`World\$`</code> 匹配行尾的 <code>World</code>
<code>[...]</code>	字符集	<code>`[aeiou]`</code> 匹配元音字母
<code>[^...]</code>	否定字符集	<code>`[^0-9]`</code> 匹配非数字
<code> </code>	或	<code>`cat dog`</code> 匹配 <code>cat</code> 或 <code>dog</code>
<code>(...)</code>	分组	<code>`(ab)+`</code> 匹配 <code>ab</code> , <code>abab</code>

NOTE

在 Java 字符串中，反斜杠需要转义，所以 `\\d` 写成 `"\\\\d"`。

3.7 性能优化

预编译 Pattern

```

1 // 错误：每次循环都编译正则
2 for (String text : texts) {
3     boolean match = text.matches("\\d+"); // 慢!
4 }
5
6 // 正确：预编译Pattern
7 Pattern pattern = Pattern.compile("\\d+");
8 for (String text : texts) {
9     Matcher matcher = pattern.matcher(text);
10    boolean match = matcher.matches(); // 快!
11 }

```

 Java

原因: `Pattern.compile()` 开销较大, 应该复用。

使用 String 的便捷方法

对于简单场景, 可以直接使用 String 的方法:

```
1 // 这些方法内部会编译Pattern, 适合偶尔使用
2 "123".matches("\\d+");
3 "hello".replaceAll("l", "L");
4 "a,b,c".split(",");
```

 Java

TIP

如果正则表达式会被多次使用, 一定要预编译 Pattern 并复用。

4 编码与字符集

4.1 UTF-8 编码

Java 内部使用 UTF-16 编码, 但外部交互常用 UTF-8:

```
1 String str = "你好世界";
2
3 // 转换为UTF-8字节数组
4 byte[] utf8Bytes = str.getBytes(StandardCharsets.UTF_8);
5
6 // 从UTF-8字节数组恢复字符串
7 String restored = new String(utf8Bytes, StandardCharsets.UTF_8);
```

 Java

CAUTION

始终显式指定字符集, 不要使用平台默认编码, 以避免跨平台问题。

4.2 常见字符集问题

乱码原因

1. 读取时使用错误的字符集
2. 写入时使用错误的字符集
3. 传输过程中编码丢失

解决方案

```
1 // 读取文件时指定编码
2 BufferedReader reader = new BufferedReader(
3     new InputStreamReader(
4         new FileInputStream("file.txt"),
5         StandardCharsets.UTF_8
6     )
7 );
```

 Java


```
8
9 // 写入文件时指定编码
10 BufferedWriter writer = new BufferedWriter(
11     new OutputStreamWriter(
12         new FileOutputStream("file.txt"),
13         StandardCharsets.UTF_8
14     )
15 );
```

5 高级字符串处理

5.1 StringJoiner (Java 8+)

更优雅的字符串连接：

```
1 // 基本用法
2 StringJoiner joiner = new StringJoiner(", ");
3 joiner.add("apple");
4 joiner.add("banana");
5 joiner.add("cherry");
6 System.out.println(joiner.toString()); // "apple, banana, cherry"
7
8 // 带前后缀
9 StringJoiner fancyJoiner = new StringJoiner(", ", "[", "]");
10 fancyJoiner.add("apple").add("banana");
11 System.out.println(fancyJoiner); // "[apple, banana]"
```

 Java

5.2 Stream API 处理字符串 (Java 8+)

```
1 List<String> words = Arrays.asList("apple", "banana", "cherry");
2
3 // 连接
4 String joined = words.stream()
5     .collect(Collectors.joining(", "));
6 // "apple, banana, cherry"
7
8 // 过滤并连接
9 String filtered = words.stream()
10     .filter(w -> w.length() > 5)
11     .collect(Collectors.joining(", "));
12 // "banana, cherry"
13
14 // 转换并连接
15 String transformed = words.stream()
16     .map(String::toUpperCase)
17     .collect(Collectors.joining(" | "));
18 // "APPLE | BANANA | CHERRY"
```

 Java

5.3 Text Blocks (Java 15+)

多行字符串字面量:

```
1 // 传统方式
2 String json = "{\n" +
3     "  \"name\": \"Alice\",\n" +
4     "  \"age\": 30\n" +
5     "}";
6
7 // Text Block
8 String jsonBlock = """
9     {
10        "name": "Alice",
11        "age": 30
12    }
13    """;
```

优势:

- 无需转义引号和换行
- 代码更清晰
- 适合 SQL、JSON、HTML 等多行文本

TIP

Text Blocks 会自动去除每行共同的缩进, 保持代码整洁。

5.4 字符串国际化

```
1 // 资源束
2 ResourceBundle bundle = ResourceBundle.getBundle("messages", Locale.CHINA);
3 String greeting = bundle.getString("greeting"); // "你好"
4
5 // 格式化
6 String message = MessageFormat.format(
7     "{0}, 欢迎! 您有 {1} 条消息",
8     "张三",
9     5
10 );
11 // "张三, 欢迎! 您有 5 条消息"
```

6 性能最佳实践

6.1 1. 避免在循环中使用字符串拼接

```
1 // 错误
2 String result = "";
3 for (int i = 0; i < 10000; i++) {
4     result += i; // 每次都创建新对象
5 }
6
7 // 正确
```

```
8  StringBuilder sb = new StringBuilder();
9  for (int i = 0; i < 10000; i++) {
10     sb.append(i);
11 }
12 String result = sb.toString();
```

6.2 2. 使用 String.intern() 谨慎

```
1 // 仅在字符串重复率很高时使用
2 String s = computeExpensiveString().intern();
```



CAUTION

`intern()` 会将字符串放入常量池，可能导致内存泄漏。只在必要时使用。

6.3 3. 预分配 StringBuilder 容量

```
1 // 如果知道大致长度
2 StringBuilder sb = new StringBuilder(1024);
```



6.4 4. 使用 char[] 处理大量字符

```
1 // 对于极端性能敏感的场景
2 char[] chars = new char[1000000];
3 // 直接操作字符数组
```



6.5 5. 合理使用 substring

```
1 // Java 7+ substring会复制字符数组
2 String sub = largeString.substring(0, 10);
3 // 如果不需要原字符串，考虑及时释放引用
4 largeString = null;
```



7 常见陷阱

7.1 陷阱 1: null 检查

```
1 String s = null;
2 // s.length(); // NullPointerException!
3
4 // 安全的方式
5 if (s != null && !s.isEmpty()) {
6     // 处理
7 }
8
9 // 或使用Objects
10 if (Objects.nonNull(s) && !s.isBlank()) {
```



```
11 // 处理
12 }
```

7.2 陷阱 2: equals vs ==

```
1 String s1 = new String("hello");
2 String s2 = new String("hello");
3
4 System.out.println(s1 == s2); // false (不同对象)
5 System.out.println(s1.equals(s2)); // true (相同内容)
```

 Java

TIP

永远使用 `equals()` 比较字符串内容，除非你明确需要比较引用。

7.3 陷阱 3: split 的正则特性

```
1 "a.b.c".split("."); // [] (.是正则元字符)
2 "a.b.c".split("\\."); // ["a", "b", "c"] (正确)
```

 Java

7.4 陷阱 4: trim vs strip

```
1 String s = "\u3000hello\u3000"; // 全角空格
2 System.out.println(s.trim().length()); // 7 (未去除)
3 System.out.println(s.strip().length()); // 5 (已去除)
```

 Java

NOTE

`strip()` 是 Java 11 引入的，能正确处理 Unicode 空白字符，推荐使用。

8 总结

Java 字符串处理的核心要点：

- **String 不可变**：线程安全，但频繁修改性能差
- **StringBuilder**：单线程高性能字符串构建
- **StringBuffer**：线程安全的字符串构建（较少使用）
- **正则表达式**：强大的模式匹配工具，注意预编译优化
- **编码处理**：始终显式指定字符集，避免乱码
- **现代特性**：Text Blocks、StringJoiner、Stream API 提升开发效率

掌握字符串处理是 Java 编程的基础，下一章将探讨集合框架的使用。

Chapter 2

集合框架

1 集合类概述

Java 集合框架用于统一管理数据集合，核心目标是：统一操作接口、屏蔽底层实现差异、提供可组合的遍历与算法能力。

INFO

图示占位：这里应有“Java 集合框架概览图（Collection / List / Set / Map 及常见实现类）”。

1.1 分类与定位

- 单列集合：以 `Collection` 为根（如 `List`、`Set`）
- 双列集合：以 `Map` 为根（键值对结构）

1.2 常见实现类速览

实现类	底层结构	核心特点	典型场景
<code>ArrayList</code>	动态数组	随机访问快，尾插高效	读多写少
<code>LinkedList</code>	双向链表	中间插删更友好	频繁插删
<code>HashSet</code>	哈希表	去重快、通常无序	成员判重
<code>TreeSet</code>	红黑树	自动排序	有序去重
<code>HashMap</code>	哈希表	查询快	通用键值存储
<code>TreeMap</code>	红黑树	按键有序	范围查询

NOTE

`List` 与 `Set` 的关键区别：

- `List`：有序、可重复、可按索引访问
- `Set`：元素唯一、通常不按索引访问

2 List 接口与实现

2.1 List 接口特点

`List` 是有序集合，允许重复元素，支持按索引访问。

方法	作用	备注
<code>add(E e)</code>	末尾添加元素	可重复
<code>get(int index)</code>	按索引读取	随机访问
<code>set(int index, E e)</code>	替换元素	返回旧值
<code>remove(int index)</code>	按索引删除	后续元素前移
<code>subList(int from, int to)</code>	截取子列表	左闭右开

Collection 基础方法（父接口）

方法	描述	注意点
<code>add</code>	添加元素	返回是否成功
<code>remove</code>	删除匹配元素	通常删首个匹配
<code>contains</code>	是否包含元素	依赖 equals
<code>size</code>	元素个数	
<code>iterator</code>	获取迭代器	遍历入口
<code>toArray</code>	转数组	

2.2 ArrayList

`ArrayList` 底层是动态数组，特点是读取快、扩容有成本。

核心特征

1. 随机访问 $O(1)$
2. 末尾追加均摊 $O(1)$
3. 中间插删通常 $O(n)$ （涉及元素搬移）

```

1 List<String> names = new ArrayList<>();
2 names.add("Alice");
3 names.add("Bob");
4 names.add(1, "Tom");
5 System.out.println(names.get(0));
6 System.out.println(names);

```

2.3 LinkedList

`LinkedList` 底层是双向链表，支持头尾快速操作，也实现了 `Deque`。

核心特征

1. 头尾插删效率高
2. 中间插删更友好（定位后修改链接）
3. 按索引随机访问慢（需要遍历）

```

1 LinkedList<Integer> queue = new LinkedList<>();

```

```
2 queue.addLast(10);
3 queue.addLast(20);
4 queue.addFirst(5);
5 System.out.println(queue.removeFirst());
6 System.out.println(queue);
```

2.4 Vector 与 Stack

`Vector` 是早期线程安全动态数组实现，整体性能通常不如 `ArrayList`。`Stack` 基于 `Vector`，遵循 LIFO。

TIP

新代码中栈结构优先考虑 `ArrayDeque`，通常更轻量、性能更好。

2.5 ArrayList 与 LinkedList 对比

对比项	ArrayList	LinkedList
底层结构	动态数组	双向链表
随机访问	快 ($O(1)$)	慢 ($O(n)$)
中间插删	慢 (搬移元素)	相对更合适
内存特性	连续空间，缓存友好	节点额外指针开销
典型场景	读多写少	频繁插删

3 Set 接口与实现

`Set` 的核心语义是“不重复”。重复判定依赖 `equals()` 与 `hashCode()`（哈希实现）或比较器（有序树实现）。

3.1 HashSet

基于哈希表，通常无序，查找/插入/删除平均接近 $O(1)$ 。

3.2 LinkedHashSet

在哈希结构上维护插入顺序，适合“去重 + 保序”。

3.3 TreeSet

基于红黑树，自动排序，复杂度通常 $O(\log n)$ 。

TreeSet 常用有序方法

方法	作用
----	----

first() / last()	取首尾元素
pollFirst() / pollLast()	取并删首尾元素
headSet(to)	小于 to 的子集
subSet(from, to)	区间子集
tailSet(from)	大于等于 from 的子集

3.4 Set 实现对比

实现	有序性	复杂度	null	场景
HashSet	通常无序	平均 $O(1)$	支持	快速去重
LinkedHashSet	按插入顺序	平均 $O(1)$	支持	去重并保序
TreeSet	按比较规则	$O(\log n)$	默认不支持（依赖比较器）	有序集合

4 Map 接口与实现

Map 保存键值映射，键唯一，值可重复。

4.1 Map 常用方法

方法	作用
put	新增/覆盖键值
get	按 key 查 value
remove	删除映射
containsKey	是否存在 key
entrySet	遍历键值对

4.2 HashMap

HashMap 是最常用实现，JDK 8 起底层可概括为“数组 + 链表 + 红黑树”。

关键参数

参数	默认值	说明
initialCapacity	16	初始桶数量（2 的幂）
loadFactor	0.75	触发扩容阈值比例
TREEIFY_THRESHOLD	8	链表树化阈值
UNTREEIFY_THRESHOLD	6	树退化阈值

MIN_TREEIFY_CAPACITY	64	最小树化容量
----------------------	----	--------

put 流程（简化）

1. 计算 key 哈希并定位桶
2. 桶为空则直接插入
3. 桶非空则处理冲突（链表/树）
4. 超阈值触发扩容或树化

```
1 Map<String, Integer> scores = new HashMap<>();
2 scores.put("Java", 95);
3 scores.put("Python", 92);
4 scores.put(null, 100);
5 System.out.println(scores.get("Java"));
```

Java

WARNING

作为 key 的自定义对象，必须正确重写 `equals()` 与 `hashCode()`，否则可能出现“放得进去、取不出来”。

图示占位：HashMap 底层结构

INFO

图示占位：这里应有“HashMap 数组-桶-链表-红黑树结构图”。

4.3 LinkedHashMap

在 HashMap 基础上维护双向链表，支持插入顺序或访问顺序。

```
1 Map<Integer, String> cache = new LinkedHashMap<>(4, 0.75f, true) {
2     @Override
3     protected boolean removeEldestEntry(Map.Entry<Integer, String> eldest) {
4         return size() > 3;
5     }
6 };
```

Java

图示占位：LRU 访问顺序

INFO

图示占位：这里应有“LinkedHashMap(accessOrder=true) 的 LRU 淘汰示意图”。

4.4 TreeMap

基于红黑树，按 key 有序，范围查询友好，复杂度通常 $O(\log n)$ 。

4.5 HashMap 1.7 vs 1.8（面试高频）

维度	JDK 1.7	JDK 1.8
----	---------	---------

结构	数组+链表	数组+链表+红黑树
插入	头插法	尾插法
扩容迁移	重新计算更重	迁移路径优化
并发风险	可能出现环链死循环	修复环链问题但仍非线程安全

5 ConcurrentHashMap 原理

5.1 为什么 HashMap 不能直接并发用

1. 并发写可能数据覆盖
2. 历史版本扩容存在环链风险
3. 复合操作（先判再改）非原子

5.2 JDK 1.7 与 1.8 的并发实现差异

版本	核心机制	锁粒度	特点
JDK 1.7	Segment 分段锁	段级	并发度受段数影响
JDK 1.8	CAS + synchronized	桶级	并发度更高

5.3 选型建议

- 高并发键值存储优先 `ConcurrentHashMap`
- 读多写少列表可考虑 `CopyOnWriteArrayList`

6 集合遍历与 fail-fast

6.1 Iterator / ListIterator / Spliterator

接口	能力	典型场景
Iterator	单向遍历 + 删除当前	通用遍历
ListIterator	双向遍历 + 就地增删改	List 增强遍历
Spliterator	可拆分遍历	Stream 并行支撑

6.2 fail-fast 机制

普通集合迭代器通常会在检测到结构性并发修改时抛 `ConcurrentModificationException`。

```

1 List<Integer> list = new ArrayList<>(List.of(1, 2, 3));
2 for (Integer x : list) {
3     if (x == 2) {
4         list.remove(x); // 可能触发 ConcurrentModificationException
5     }

```

 Java

```
6 }
```

NOTE

fail-fast 是“尽早暴露误用”的机制，不等价于线程安全保障。

6.3 安全修改建议

1. 遍历中删除优先 `Iterator.remove()`
2. 条件删除优先 `removeIf()`
3. 并发场景改用并发容器

7 多线程集合类

7.1 并发集合速查

集合	特点	适用场景
<code>ConcurrentHashMap</code>	高并发键值读写	缓存、路由表
<code>CopyOnWriteArrayList</code>	读无锁，写复制	读多写少配置
<code>ConcurrentSkipListMap</code>	并发有序映射	有序并发访问
<code>LinkedBlockingQueue</code>	阻塞 FIFO	生产者-消费者
<code>ArrayBlockingQueue</code>	有界阻塞队列	限流缓冲

7.2 普通集合 vs 并发集合

场景	推荐
单线程	<code>ArrayList</code> / <code>HashMap</code>
高并发读写	<code>ConcurrentHashMap</code>
读多写少	<code>CopyOnWriteArrayList</code>
任务队列	<code>BlockingQueue</code> 系列

8 深拷贝

8.1 深拷贝与浅拷贝

- 浅拷贝：复制外层对象，内部引用共享
- 深拷贝：递归复制内部引用对象

8.2 常见实现方式

构造函数复制

优点是可控、类型安全；缺点是对象层级深时代码冗长。

clone() 重写

需实现 `Cloneable`，并在引用字段上继续深拷贝。

序列化/反序列化复制

可借助 Commons Lang、Gson、Jackson 等，开发效率高，但需关注性能与类型约束。

方式	优点	缺点	适用
构造函数	显式可控	模板代码多	小中型对象
clone	JDK 原生	易误用浅拷贝	中等复杂度
序列化	实现快	性能开销较大	工具化场景

9 补充说明

9.1 正确重写 equals 与 hashCode

放入哈希集合（`HashSet` / `HashMap`）的自定义对象，必须满足：

- 1. `equals` 相等的对象，`hashCode` 必须相等
- 2. 同一次运行期间，不变字段对应的 `hashCode` 应保持稳定

```
1 public class Person {
2     private String name;
3     private int age;
4
5     @Override
6     public boolean equals(Object o) {
7         if (this == o) return true;
8         if (o == null || getClass() != o.getClass()) return false;
9         Person p = (Person) o;
10        return age == p.age && Objects.equals(name, p.name);
11    }
12
13    @Override
14    public int hashCode() {
15        return Objects.hash(name, age);
16    }
17 }
```

9.2 集合选型经验法则

需求	首选
按索引随机访问	ArrayList

频繁头尾操作	ArrayDeque / LinkedList
快速去重	HashSet
去重并保序	LinkedHashSet
有序键值查询	TreeMap
高并发键值	ConcurrentHashMap

9.3 时间复杂度参考

操作	ArrayList	LinkedList	HashMap	TreeMap
添加	均摊 $O(1)$	$O(1)$	平均 $O(1)$	$O(\log n)$
删除	$O(n)$	按索引/值通常 $O(n)$	平均 $O(1)$	$O(\log n)$
查找	$O(1)$ 按索引	$O(n)$	平均 $O(1)$	$O(\log n)$

9.4 不可变集合与只读视图

很多同学会把“不可变集合”和“只读视图”混为一谈，二者并不等价：

1. `List.of()` / `Set.of()` / `Map.of()` 返回真正不可变集合
2. `Collections.unmodifiableList(x)` 返回的是“只读视图”，底层 `x` 变了，视图也会变

```

1 List<String> src = new ArrayList<>(List.of("A", "B"));
2 List<String> view = Collections.unmodifiableList(src);
3 src.add("C");
4 System.out.println(view); // [A, B, C]
5
6 List<String> imm = List.of("X", "Y");
7 // imm.add("Z"); // UnsupportedOperationException

```

WARNING

对外暴露集合时，若要求“状态绝对不可被修改”，优先返回拷贝后的不可变集合，而不是只读视图。

9.5 Map 遍历性能建议

遍历 `Map` 时通常优先 `entrySet()`，避免“先遍历 key 再 `get(value)`”的重复查找开销。

```

1 // 推荐
2 for (Map.Entry<String, Integer> e : map.entrySet()) {
3     String k = e.getKey();
4     Integer v = e.getValue();
5 }
6
7 // 次优（会重复哈希查找）
8 for (String k : map.keySet()) {
9     Integer v = map.get(k);
10 }

```

9.6 常见易错点清单

1. 在 `for-each` 中直接调用集合自身 `remove()`，触发 `ConcurrentModificationException`
2. 自定义对象做 `HashMap` key 却未同时重写 `equals/hashCode`
3. 误把 `LinkedList` 当作高性能随机访问容器
4. 并发场景继续使用 `HashMap` / `ArrayList` 做共享可变状态
5. 误以为 `unmodifiable*` 返回的是“深层不可变对象”

10 总结

掌握集合框架的关键不是背类名，而是理解这几个维度：

1. 底层结构（数组/链表/哈希/树/跳表）
2. 复杂度特征（时间与空间）
3. 语义特征（有序、可重复、null 支持）
4. 并发特征（线程安全与迭代一致性）

一句话：先看访问模式，再选数据结构，最后看并发与可维护性。

Chapter 3

异常处理

1 异常概述

Java 中的异常处理机制与 C# 的核心思想类似，都是通过 `try-catch-finally` 捕获和处理运行问题。但 Java 在类型体系上更严格，尤其是“受检异常”会被编译器强制检查。

1.1 基本语法结构

```
1 try {  
2     // 需要进行异常捕获的代码块  
3 } catch (Exception ex) {  
4     // 捕获异常信息；可定义多个 catch，分别处理不同异常  
5 } finally {  
6     // 无论是否发生异常，都会尝试执行  
7 }
```

1.2 异常体系速览

异常根类型是 `java.lang.Throwable`，其下分为两条主线：

- `Error`：系统级严重错误，通常不建议业务代码处理
- `Exception`：程序可处理异常

`Exception` 下又分为：

- `RuntimeException` 及其子类：非检查性异常（unchecked）
- 其他 `Exception` 子类：检查性异常（checked）

NOTE

Java 的核心设计是：可预期且可恢复的问题，尽量在编译期逼迫开发者显式处理。

1.3 常见异常一览

非检查性异常（`RuntimeException`）

异常	描述
<code>ArithmeticException</code>	整数除零等算术非法操作
<code>ArrayIndexOutOfBoundsException</code>	数组下标越界

ClassCastException	类型强转失败
IllegalArgumentException	传入参数不合法
IllegalStateException	对象状态不符合当前操作要求
NegativeArraySizeException	创建了负长度数组
NullPointerException	在需要对象处使用了 null
NumberFormatException	字符串无法转换为数值
UnsupportedOperationException	调用了不支持的操作

检查性异常 (Checked Exception)

异常	描述
ClassNotFoundException	动态加载类时未找到目标类
CloneNotSupportedException	对象未实现 Cloneable 却调用 clone
IllegalAccessException	访问权限不满足
InstantiationException	试图实例化抽象类/接口
InterruptedException	线程被中断
NoSuchFieldException	反射访问字段不存在
NoSuchMethodException	反射访问方法不存在

1.4 Throwable 常用方法

方法	说明
getMessage()	返回异常简要描述信息
getCause()	返回导致当前异常的根因异常
toString()	返回异常类型 + 消息
printStackTrace()	打印完整堆栈
getStackTrace()	返回堆栈数组供程序分析
fillInStackTrace()	填充当前堆栈信息

2 异常类型与层次结构

2.1 Throwable / Error / Exception

`Throwable` 是“可被抛出对象”的统一抽象。只有 `Throwable` 或其子类实例，才可以被 `throw` 抛出。

系统错误 (Error)

Error 一般由 JVM 抛出，表示严重运行问题。常见如：

类	可能原因
LinkageError	类依赖在编译后发生不兼容变更
VirtualMachineError	JVM 运行资源不足或内部错误
OutOfMemoryError	内存耗尽
NoClassDefFoundError	类加载失败
StackOverflowError	递归过深导致栈溢出

WARNING

Error 通常不属于可恢复业务异常。生产系统应以“告警 + 降级 + 保护性终止”为主，而不是继续执行业务逻辑。

运行时异常 (RuntimeException)

运行时异常通常反映程序逻辑错误，编译器不会强制捕获或声明。

- 常见场景：空引用访问、参数不合法、数组越界、类型转换失败
- 处理策略：优先修复根因，必要时在边界层统一兜底

编译时异常 (Checked Exception)

除 **RuntimeException** 及其子类外，大多数 **Exception** 都是受检异常，必须显式处理：

1. 使用 **try-catch** 捕获
2. 或在方法签名中用 **throws** 继续上抛

TIP

“可恢复并可预期”的问题，适合用 checked exception；“编程错误”通常属于 unchecked exception。

3 异常处理机制

3.1 声明异常 (throws)

方法可在签名中声明可能抛出的受检异常：

```
1 public void readFile() throws IOException {  
2     // ...  
3 }  
4  
5 public void process() throws IOException, SQLException {  
6     // ...  
7 }
```

重写方法时的 throws 规则

1. 子类重写方法抛出的受检异常，不能比父类更宽
2. 可以抛出父类已声明异常的子类
3. 父类方法未声明受检异常时，子类不能新增受检异常声明

3.2 抛出异常 (throw)

在检测到非法状态时主动抛出异常：

```
1 if (age < 0) {  
2     throw new IllegalArgumentException("年龄不能为负数");  
3 }
```

 Java

throw 与 throws 区别

- `throw`：抛出一个具体异常对象
- `throws`：声明方法可能抛出的异常类型

3.3 捕获异常 (try-catch-finally)

推荐按“最具体异常到最通用异常”的顺序编写多重 catch：

```
1 try {  
2     String s = args[0];  
3     int n = Integer.parseInt(s);  
4     int r = 10 / n;  
5 } catch (ArrayIndexOutOfBoundsException e) {  
6     System.out.println("缺少参数");  
7 } catch (NumberFormatException e) {  
8     System.out.println("参数必须是数字");  
9 } catch (ArithmeticException e) {  
10    System.out.println("除数不能为零");  
11 } catch (Exception e) {  
12    System.out.println("未知错误: " + e.getMessage());  
13 } finally {  
14    System.out.println("资源收尾");  
15 }
```

 Java

finally 执行时机

1. try 正常结束后执行 finally
2. try 抛异常并被 catch 处理后执行 finally
3. try 抛异常未被当前方法处理，也会先执行 finally 再向上抛
4. try/catch 中出现 return，finally 仍会在方法返回前执行

return 与 finally 的交互

```
1 public static int test1() {  
2     try {  
3         return 1;  
4     }
```

 Java

```
4    } finally {  
5        System.out.println("finally");  
6    }  
7 }  
8  
9 public static int test2() {  
10     try {  
11         return 1;  
12     } finally {  
13         return 2; // 会覆盖 try 中的返回值（不推荐）  
14     }  
15 }
```

DANGER

不建议在 finally 中写 return。它会覆盖原返回值，甚至吞掉原始异常，导致排障困难。

finally 可能不执行的极端情况

- 调用 `System.exit()` 直接终止 JVM
- JVM 崩溃或严重错误导致进程中止
- 线程被强制终止（历史 API，如 `Thread.stop()`）

3.4 try-with-resources (Java 7+)

对于实现 `AutoCloseable` 的资源，优先使用自动关闭语法：

```
1 try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {  
2     String line = br.readLine();  
3     System.out.println(line);  
4 } catch (IOException e) {  
5     e.printStackTrace();  
6 }
```

 Java

suppressed 异常说明（易忽略）

当 `try` 块和资源关闭过程都抛异常时：

1. 主异常通常是 `try` 块中的异常
2. 关闭资源时产生的异常会被放入 `getSuppressed()`

```
1 try (MyResource r = new MyResource()) {  
2     throw new RuntimeException("主异常");  
3 } catch (Exception e) {  
4     System.out.println(e.getMessage()); // 主异常  
5     for (Throwable s : e.getSuppressed()) {  
6         System.out.println("suppressed: " + s.getMessage());  
7     }  
8 }
```

 Java**NOTE**

排查线上问题时，不要只看 `getMessage()`；应同时检查 `getCause()` 与 `getSuppressed()`，否则可能漏掉资源关闭阶段的关键异常。

4 自定义异常

4.1 何时使用自定义异常

当内置异常无法准确表达业务语义时，应定义业务异常类。例如：订单状态非法、余额不足、重复提交。

4.2 自定义异常示例

```
1 public class MyException extends Exception {
2     public MyException(String message) {
3         super(message);
4     }
5 }
6
7 try {
8     int a = -1;
9     if (a < 0) throw new MyException("a 不能小于 0");
10    int[] array = new int[a];
11 } catch (MyException e) {
12     System.out.println(e.getMessage());
13 } finally {
14     System.out.println("异常捕获结束");
15 }
```

命名与设计建议

1. 异常类名统一以 `Exception` 结尾
2. 需强制调用方处理时继承 `Exception`
3. 业务运行时错误可继承 `RuntimeException`
4. 保留异常链，优先使用带 `cause` 的构造方法

5 最佳实践与反模式

5.1 推荐实践

1. 只捕获你能处理的异常
2. 不要吞异常；至少记录日志
3. 保留异常链，不要丢失原始堆栈
4. 优先用具体异常类型，而不是泛化到 `Exception`
5. 资源释放优先使用 `try-with-resources`

`InterruptedException` 处理规范

对于 `InterruptedException`，常见正确姿势是“恢复中断标记”或“继续向上抛出”：

```
1 try {
```

```
2 Thread.sleep(1000);
3 } catch (InterruptedException e) {
4     Thread.currentThread().interrupt(); // 恢复中断标记
5     throw new RuntimeException("线程被中断", e);
6 }
```

WARNING

不要简单吞掉 `InterruptedException`。这会破坏线程协作协议，导致线程池任务无法按预期停止。

异常日志建议

记录异常时应包含：

1. 业务上下文（请求 ID、用户 ID、关键参数）
2. 异常类型 + 异常消息 + 完整堆栈
3. 关键分支状态（重试次数、远端响应码）

并避免：

- 只打印 `e.getMessage()` 不打印堆栈
- 重复多层打印同一异常造成日志噪声
- 将密码、令牌等敏感信息写入日志

分层异常转换（Service / Controller）

推荐在分层架构中做“边界转换”：

1. DAO 层抛技术异常（SQL/IO）
2. Service 层转换为业务异常并补充语义
3. Controller 层统一映射为错误码与对外响应

```
1 try {
2     userRepository.save(user);
3 } catch (SQLException e) {
4     throw new UserOperationException("用户保存失败", e);
5 }
```

 Java**TIP**

统一异常映射（例如 Web 全局异常处理器）能显著提升接口一致性，减少散落在控制器中的重复 `try-catch`。

5.2 反模式示例

```
1 // 反模式：吞异常
2 try {
3     doSomething();
4 } catch (Exception e) {
5     // nothing
6 }
7
8 // 反模式：抛新异常但丢失原始堆栈
9 try {
10    doSomething();
```

 Java

```
11 } catch (IOException e) {  
12     throw new RuntimeException(e.getMessage());  
13 }  
14  
15 // 正确: 保留 cause  
16 try {  
17     doSomething();  
18 } catch (IOException e) {  
19     throw new RuntimeException("处理失败", e);  
20 }
```

6 面试与实战高频

6.1 Checked vs Unchecked

维度	Checked Exception	Unchecked Exception
编译器检查	强制检查	不强制检查
处理要求	必须捕获或声明	可自行选择
典型类型	IOException、SQLException	NullPointerException、IllegalArgumentException
设计意图	可恢复、可预期问题	编程错误或非法状态

6.2 高频问答

try 里 return, finally 会执行吗

会执行。finally 在方法真正返回前执行。

try/catch/finally 都有 return, 最终返回哪个

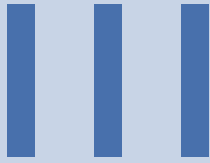
finally 中的 return 会覆盖前面的返回值。

finally 中修改变量为何有时生效有时不生效

- 基本类型: return 时值已拷贝, finally 修改通常不影响返回
- 引用类型: return 的是引用, finally 修改对象内容通常可见

TIP

生产代码里尽量避免在 finally 中修改返回路径。可读性差、风险高、容易引入隐蔽 bug。



高级特性与函数式编程

IV

并发编程

4.	并发基础与问题模型 ·····	41
5.	线程安全与同步机制 ·····	46
6.	并发协作与线程池 ·····	52
7.	Kotlin 协程基础 ·····	67
8.	协程进阶与异步流 ·····	81
9.	并发诊断与综合实战 ·····	82

Chapter 4

并发基础与问题模型

1 进程与线程、并发 vs 并行

1.1 进程与线程的区别

进程和线程是操作系统调度的基本单位，但二者在资源和隔离性上存在本质差异：

- **进程 (Process)**：操作系统分配资源的基本单位，拥有独立的地址空间（堆、方法区）、文件句柄和信号量。进程间通信（IPC）需要借助管道、消息队列、共享内存等机制。
- **线程 (Thread)**：CPU 调度的最小单位，同一进程内的线程共享堆内存和方法区，但每个线程拥有独立的程序计数器（PC）、虚拟机栈和本地方法栈。

维度	进程	线程
资源拥有	独立地址空间	共享进程资源
通信方式	IPC 机制	直接读写共享变量
开销	创建/切换成本高	创建/切换成本低
隔离性	进程间隔离	线程间不隔离

1.2 并发与并行的区分

- **并发 (Concurrency)**：逻辑上的同时发生。多个任务在同一个时间段内交替执行，通过 CPU 时间片轮转实现。单核 CPU 即可实现并发。
- **并行 (Parallelism)**：物理上的同时发生。多个任务在同一时刻真正同时执行，需要多核 CPU 支持。

TIP

并发关注的是任务调度的结构（设计层面），并行关注的是任务执行的物理方式（实现层面）。高并发通常需要良好的并行能力作为支撑。

1.3 上下文切换的成本

当 CPU 在不同线程间切换时，需要保存当前线程的上下文（寄存器、程序计数器、栈指针等）并加载新线程的上下文。

成本类型	说明
CPU 时间	每次切换需要数百到数千个 CPU 时钟周期
缓存失效	线程切换导致 L1/L2/L3 缓存命中率下降

调度延迟

内核调度器介入带来的延迟

WARNING

线程并非越多越好。过多的线程会导致频繁的上下文切换，反而降低系统吞吐量。线程数的设置需要根据任务类型（CPU 密集型 vs IO 密集型）和硬件配置来权衡。

2 Java 线程生命周期与操作

2.1 线程状态流转

Java 线程的生命周期由 `java.lang.Thread.State` 枚举定义：

状态	JVM 状态	操作系统状态	说明
NEW	NEW	初始	线程已创建但未调用 <code>`start()`</code>
RUNNABLE	RUNNABLE	Ready / Running	可运行状态，可能正在等待 CPU 或正在执行
BLOCKED	BLOCKED	Blocked	等待获取监视器锁（synchronized） 无限期等待，需其他线程显式唤醒
WAITING	WAITING	Waiting	<code>`wait()`</code> 、 <code>`join()`</code> 、 <code>`LockSupport.park()`</code>
TIMED_WAITING	TIMED_WAITING	Timed Waiting	限期等待（ <code>`sleep(ms)`</code> 、 <code>`wait(ms)`</code> 、 <code>`join(ms)`</code> 、 <code>`LockSupport.parkNanos()`</code> ）
TERMINATED	TERMINATED	Terminated	线程执行完毕或抛出未捕获异常

2.2 线程创建方式

Java 中创建线程有三种主流方式：

继承 Thread 类

```
1 public class MyThread extends Thread {
2     @Override
3     public void run() {
4         System.out.println("Thread running: " + Thread.currentThread().getName());
5     }
6 }
7
8 // 使用
9 new MyThread().start();
```

实现 Runnable 接口（推荐）

```
1 public class MyRunnable implements Runnable {
2     @Override
3     public void run() {
4         System.out.println("Runnable running: " + Thread.currentThread().getName());
5     }
6 }
7
8 // 使用
9 new Thread(new MyRunnable(), "MyThread").start();
10
11 // Lambda 简化 (Java 8+)
12 new Thread(() -> System.out.println("Lambda thread"), "LambdaThread").start();
```

实现 Callable 接口

```
1 public class MyCallable implements Callable<Integer> {
2     @Override
3     public Integer call() throws Exception {
4         return 42;
5     }
6 }
7
8 // 使用（需配合 FutureTask）
9 FutureTask<Integer> task = new FutureTask<>(new MyCallable());
10 new Thread(task).start();
11 Integer result = task.get(); // 阻塞等待结果
```

NOTE

推荐使用 `Runnable` 接口，因为 Java 不支持多继承，继承 `Thread` 会导致该类无法继承其他类。而 `Runnable` 接口还可以配合线程池、`CompletableFuture` 等使用。

2.3 守护线程

守护线程（Daemon Thread）为用户线程提供服务。当 JVM 中所有用户线程都结束时，守护线程会自动终止，JVM 也会随之退出。

```
1 Thread daemonThread = new Thread(() -> {
2     while (true) {
3         // 后台任务
4     }
5 });
6 daemonThread.setDaemon(true);
7 daemonThread.start();
```

TIP

典型的守护线程：Java 垃圾回收线程（GC）、定时调度线程等。设置守护线程时应在 `start()` 之前调用 `setDaemon(true)`。

2.4 线程中断机制

中断是 Java 线程间协作的一种方式，而非强制终止。正确的中断处理流程：

```
1 public class InterruptExample implements Runnable {
2     @Override
3     public void run() {
4         while (!Thread.currentThread().isInterrupted()) {
5             try {
6                 // 业务逻辑
7                 Thread.sleep(100);
8             } catch (InterruptedException e) {
9                 // 收到中断信号后，应退出循环或向上传播
10                Thread.currentThread().interrupt();
11                break;
12            }
13        }
14    }
15 }
```

CAUTION

不要在 `run()` 方法中捕获 `InterruptedException` 后什么都不做，这会清除中断状态。正确的做法是：1) 重新设置中断状态 `Thread.currentThread().interrupt()`；2) 退出循环或向上传播异常。

3 并发核心问题：三性模型

在应用层面，并发编程面临三个核心问题，理解它们是编写线程安全代码的基础。

3.1 原子性 (Atomicity)

原子性指一个或多个操作在执行过程中不会被其他线程干扰，要么全部执行成功，要么全部不执行。

- 基本类型读写：除 `long` 和 `double` 外的赋值操作是原子的。
- 复合操作：`i++`、`count = count + 1` 等不是原子操作。

Algorithm

原子性问题的典型案例：

- 线程 A 读取 `count = 5`
- 线程 B 读取 `count = 5`
- 线程 A 计算 `5 + 1 = 6`，写回
- 线程 B 计算 `5 + 1 = 6`，写回
- 期望结果：7，实际结果：6（丢失了一次更新）

3.2 可见性 (Visibility)

可见性问题指一个线程对共享变量的修改，其他线程能否立即看到。

- 成因：CPU 多级缓存导致。每个 CPU 核心有自己的 L1/L2/L3 缓存，线程可能读取到缓存中的旧值而非主存中的最新值。

WARNING

即使在单核 CPU 上，也存在可见性问题（由于编译器和 CPU 的指令重排序）。多核 CPU 下问题更加突出。

3.3 有序性 (Ordering)

程序代码的执行顺序应当与代码书写顺序一致，但由于编译器和处理器的优化，会发生指令重排序。

- **编译重排序**：编译器在不改变单线程语义的前提下调整指令顺序。
- **指令重排序**：处理器采用指令级并行技术（ILP）优化执行顺序。
- **内存重排序**：由于 CPU 缓存一致性协议导致看起来“顺序错乱”的内存访问。

NOTE

Java 内存模型 (JMM) 和 Happens-Before 规则的深入内容将在 Part 6: JVM 底层原理中详细讲解。本节聚焦于应用层的直观理解。

Chapter 5

线程安全与同步机制

当多个线程同时访问同一可变状态时，如果不进行适当同步，就会产生线程安全问题。

1 synchronized 与 Lock

1.1 synchronized 关键字

`synchronized` 是 Java 内置的同步机制，基于对象的监视器锁（Monitor Lock）实现。

三种使用形式

```
1 // 1. 修饰实例方法（锁定当前实例对象）
2 public synchronized void method() {
3     // 同一时刻只有一个线程能执行此方法
4 }
5
6 // 2. 修饰静态方法（锁定当前类的 Class 对象）
7 public static synchronized void staticMethod() {
8     // 同一时刻只有一个线程能执行此方法
9 }
10
11 // 3. 修饰代码块（锁定指定对象）
12 public void blockMethod() {
13     synchronized (this) {
14         // 锁定当前实例
15     }
16     synchronized (Lock.class) {
17         // 锁定 Class 对象
18     }
19 }
```

锁的升级过程

阶段	锁类型	特点
无锁	偏向锁	首次获取时记录线程 ID，后续进入同步块无需任何同步
偏向锁	轻量级锁	有其他线程竞争时升级，CAS 自旋争取
轻量级锁	重量级锁	自旋失败后升级，阻塞等待

TIP

偏向锁在 Java 15+ 已被废弃，默认关闭。日常开发中无需过度关注锁升级细节，理解 `synchronized` 的基本语义即可。

1.2 Lock 接口

`java.util.concurrent.locks.Lock` 提供了比 `synchronized` 更灵活的锁操作。

```
1 Lock lock = new ReentrantLock();
2 lock.lock();
3 try {
4     // 临界区代码
5 } finally {
6     lock.unlock(); // 必须在 finally 中释放
7 }
```

 Java

ReentrantLock 特性

- 可重入：同一线程可以多次获取同一把锁。
- 公平/非公平：公平锁按等待顺序获取，非公平锁允许插队（默认）。
- 尝试获取锁：`tryLock()`、`tryLock(timeout)`。
- 条件变量：`newCondition()` 创建多个条件等待集。

```
1 ReentrantLock lock = new ReentrantLock(true); // 公平锁
2 Condition condition = lock.newCondition();
3
4 // 等待条件
5 lock.lock();
6 try {
7     while (!conditionMet) {
8         condition.await(); // 释放锁并等待
9     }
10 } finally {
11     lock.unlock();
12 }
13
14 // 唤醒条件
15 lock.lock();
16 try {
17     conditionMet = true;
18     condition.signalAll();
19 } finally {
20     lock.unlock();
21 }
```

 Java**CAUTION**

使用 `Lock` 时必须确保在 `finally` 块中释放锁，否则如果临界区抛出异常，锁将永远无法释放，导致死锁。

1.3 volatile 关键字

`volatile` 是轻量级的同步机制，保证可见性和有序性，但不保证原子性。

Java

```
1 volatile boolean flag = false;
2
3 // 线程 A
4 flag = true;
5
6 // 线程 B
7 while (!flag) {
8     // 能立即看到 flag 的变化
9 }
```

NOTE

`volatile` 的实现原理：写入时立即刷新到主存，读取时从主存获取。禁止指令重排序（通过内存屏障实现）。

1.4 Happens-Before 规则

Happens-Before 是 Java 内存模型（JMM）定义的操作偏序关系，是判断并发安全性的理论依据。

规则	含义
程序顺序规则	同一线程中，前面的操作 happens-before 后面的操作
监视器锁规则	解锁 happens-before 后续的加锁
volatile 变量规则	写 happens-before 后续的读
线程启动规则	Thread.start() happens-before 被启动线程的任何操作
线程终止规则	线程的所有操作 happens-before 其他线程检测到该线程终止
传递性	A happens-before B, B happens-before C, 则 A happens-before C

WARNING

理解 Happens-Before 规则是分析并发代码正确性的关键。很多“看起来正确”的并发代码实际上存在问题，需要通过这些规则来验证。

2 CAS 与原子类

2.1 CAS 原理

CAS（Compare-And-Swap）是一种无锁算法，通过硬件指令实现原子操作。

Java

```
1 // 伪代码
2 boolean compareAndSwap(Object obj, long offset, Object expected, Object newValue) {
3     Object current = obj.get(offset); // 读取当前值
4     if (current == expected) {
5         obj.set(offset, newValue); // 如果相等则更新
6         return true;
7     }
```



```
8     return false;
9 }
```

CAS 包含三个操作数：内存位置（V）、预期原值（A）和新值（B）。只有当 V 的值等于 A 时，才将 V 的值更新为 B。

CAS 的问题

- ABA 问题：值从 A 变为 B 再变回 A，CAS 无法检测。
- 循环开销：自旋 CAS 如果长时间不成功，会占用 CPU。
- 只能保证一个变量：无法对多个变量进行原子操作。

2.2 原子类 (java.util.concurrent.atomic)

Java 提供了一系列原子类，基于 CAS 实现了高效的原子操作。

原子整数

```
1  AtomicInteger counter = new AtomicInteger(0);
2
3  // 原子递增
4  counter.incrementAndGet(); // ++i, 返回新值
5  counter.getAndIncrement(); // i++, 返回旧值
6  counter.addAndGet(5);      // i += 5
7
8  // CAS 更新
9  counter.compareAndSet(10, 20); // 如果当前值是 10, 则更新为 20
10
11 // 原子更新
12 counter.updateAndGet(x -> x * 2); // 函数式更新
```

 Java

原子引用

```
1  AtomicReference<User> userRef = new AtomicReference<>(new User("Alice"));
2
3  // 原子替换
4  userRef.set(new User("Bob"));
5
6  // CAS 更新
7  userRef.compareAndSet(oldUser, newUser);
8
9  // 函数式更新
10 userRef.updateAndGet(u -> new User(u.name.toUpperCase()));
```

 Java

ABA 问题解决

```
1  // 使用 AtomicStampedReference 添加版本号
2  AtomicStampedReference<Integer> ref = new AtomicStampedReference<>(100, 0);
3
4  int stamp = ref.getStamp();
5  ref.compareAndSet(100, 101, stamp, stamp + 1);
```

 Java

```

6
7 // 或使用 AtomicMarkableReference (布尔标记)
8 AtomicMarkableReference<Integer> ref2 = new AtomicMarkableReference<>(100, false);

```

TIP

原子类适合在低并发场景下替代锁，实现简单且性能优秀。但在高并发、竞争激烈的场景下，需要评估自旋带来的 CPU 消耗。

3 线程安全集合与并发容器

Java 并发包（java.util.concurrent）提供了多种线程安全的集合类，适用于不同场景。

3.1 ConcurrentHashMap

并发哈希表是使用最广泛的并发容器，采用分段锁（Java 7）或 CAS + synchronized（Java 8+）实现。

```

1 ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();
2
3 // 原子操作
4 map.putIfAbsent("key", 1); // 仅当 key 不存在时插入
5 map.computeIfAbsent("key", k -> computeValue(k)); // 原子计算并插入
6 map.merge("key", 1, Integer::sum); // 原子合并
7
8 // 批量操作（弱一致性）
9 map.forEach(1, (k, v) -> System.out.println(k + ":" + v));
10 map.keys(); // 返回迭代器

```



操作	普通 HashMap	ConcurrentHashMap
线程安全	否	是
null 键/值	允许	不允许
迭代器	fail-fast	弱一致性
锁粒度	全局锁	分段锁/CAS

3.2 CopyOnWrite 系列

CopyOnWriteArrayList

写时复制机制：每次修改操作都会创建底层数组的新副本。

```

1 CopyOnWriteArrayList<String> list = new CopyOnWriteArrayList<>();
2
3 list.add("element"); // 每次添加都复制整个数组
4 list.remove("element");
5 list.get(0); // 读操作无需加锁

```



NOTE

适合读多写少的场景（如监听器列表、配置信息）。写操作开销大，不适合频繁修改的数据。

CopyOnWriteArraySet

基于 CopyOnWriteArrayList 实现的线程安全 Set。

```
1 CopyOnWriteArraySet<String> set = new CopyOnWriteArraySet<>();
```



3.3 其他并发容器

容器	特点	适用场景
ConcurrentLinkedQueue	无界、非阻塞队列	高并发队列操作
LinkedBlockingQueue	可选有界、阻塞队列	生产者-消费者模式
ArrayBlockingQueue	有界、阻塞队列	需要容量限制的场景
ConcurrentSkipListMap	线程安全跳表实现	需要有序且线程安全的 Map
ConcurrentSkipListSet	线程安全跳表实现	需要有序且线程安全的 Set

WARNING

切勿在遍历并发容器时使用普通的 for-each 或迭代器进行修改。应当使用容器自身提供的原子操作（如 forEach、computeIfAbsent 等）。

Chapter 6

并发协作与线程池

在实际应用中，线程之间需要协作完成任务，而线程池则是管理线程资源的核心机制。本章深入探讨线程通信、线程池原理和 JUC 工具类。

NOTE

线程池的核心理念是资源复用和任务解耦，避免频繁创建销毁线程的开销，提高系统性能和稳定性。

1 线程通信

线程通信是多线程协作的基础，Java 提供了多种机制。

1.1 wait/notify 机制

基于 Object 类的 wait()和 notify()方法实现线程间通信。

基本原理

```
1 public class WaitNotifyExample {
2     private final Object lock = new Object();
3     private boolean condition = false;
4
5     public void waitForCondition() throws InterruptedException {
6         synchronized (lock) {
7             while (!condition) { // 必须用while, 不能用if
8                 System.out.println(Thread.currentThread().getName() + " waiting...");
9                 lock.wait(); // 释放锁, 进入等待队列
10            }
11            System.out.println(Thread.currentThread().getName() + " notified!");
12        }
13    }
14
15    public void setCondition() {
16        synchronized (lock) {
17            condition = true;
18            lock.notify(); // 唤醒一个等待线程
19            // lock.notifyAll(); // 唤醒所有等待线程
20        }
21    }
22 }
```

关键点：

1. 必须在 `synchronized` 块中调用
2. 使用 `while` 循环检查条件（防止虚假唤醒）
3. `wait()`释放锁，其他线程可以获取锁
4. `notify()`不立即释放锁，要等 `synchronized` 块执行完

CAUTION

永远使用 `while` 循环而不是 `if` 来检查条件，因为存在虚假唤醒（Spurious Wakeup）的可能。

生产者-消费者模型

```
1  public class ProducerConsumer {
2      private final Queue<Integer> buffer = new LinkedList<>();
3      private final int capacity = 10;
4
5      public void produce() throws InterruptedException {
6          int value = 0;
7          while (true) {
8              synchronized (this) {
9                  // 缓冲区满，等待
10                 while (buffer.size() == capacity) {
11                     System.out.println("Buffer full, producer waiting...");
12                     wait();
13                 }
14
15                 // 生产数据
16                 buffer.add(value);
17                 System.out.println("Produced: " + value);
18                 value++;
19
20                 // 通知消费者
21                 notifyAll();
22             }
23
24             Thread.sleep(100); // 模拟生产时间
25         }
26     }
27
28     public void consume() throws InterruptedException {
29         while (true) {
30             synchronized (this) {
31                 // 缓冲区空，等待
32                 while (buffer.isEmpty()) {
33                     System.out.println("Buffer empty, consumer waiting...");
34                     wait();
35                 }
36
37                 // 消费数据
38                 int value = buffer.poll();
39                 System.out.println("Consumed: " + value);
40
41                 // 通知生产者
42                 notifyAll();
```

```
43         }
44
45         Thread.sleep(150); // 模拟消费时间
46     }
47 }
48 }
```

TIP

生产者-消费者模式是经典的并发模式，实现了生产和消费的解耦。

1.2 Condition 接口

Condition 提供了更灵活的线程等待/通知机制。

基本用法

```
1  import java.util.concurrent.locks.*;
2
3  public class ConditionExample {
4      private final ReentrantLock lock = new ReentrantLock();
5      private final Condition notFull = lock.newCondition();
6      private final Condition notEmpty = lock.newCondition();
7      private final Queue<Integer> buffer = new LinkedList<>();
8      private final int capacity = 10;
9
10     public void produce(int value) throws InterruptedException {
11         lock.lock();
12         try {
13             while (buffer.size() == capacity) {
14                 System.out.println("Buffer full, waiting...");
15                 notFull.await(); // 在notFull条件上等待
16             }
17
18             buffer.add(value);
19             System.out.println("Produced: " + value);
20
21             notEmpty.signal(); // 通知notEmpty条件的等待线程
22         } finally {
23             lock.unlock();
24         }
25     }
26
27     public int consume() throws InterruptedException {
28         lock.lock();
29         try {
30             while (buffer.isEmpty()) {
31                 System.out.println("Buffer empty, waiting...");
32                 notEmpty.await(); // 在notEmpty条件上等待
33             }
34
35             int value = buffer.poll();
```

```
36         System.out.println("Consumed: " + value);
37
38         notFull.signal(); // 通知notFull条件的等待线程
39         return value;
40     } finally {
41         lock.unlock();
42     }
43 }
44 }
```

优势:

- 可以有多个条件变量
- 可以精确唤醒特定条件的线程
- 支持中断和超时

多条件示例

```
1 public class BoundedBuffer {
2     private final ReentrantLock lock = new ReentrantLock();
3     private final Condition notFull = lock.newCondition();
4     private final Condition notEmpty = lock.newCondition();
5     private final Object[] items = new Object[100];
6     private int count, putIndex, takeIndex;
7
8     public void put(Object x) throws InterruptedException {
9         lock.lock();
10        try {
11            while (count == items.length)
12                notFull.await();
13
14            items[putIndex] = x;
15            putIndex = (putIndex + 1) % items.length;
16            ++count;
17
18            notEmpty.signal(); // 只唤醒消费者
19        } finally {
20            lock.unlock();
21        }
22    }
23
24    public Object take() throws InterruptedException {
25        lock.lock();
26        try {
27            while (count == 0)
28                notEmpty.await();
29
30            Object x = items[takeIndex];
31            takeIndex = (takeIndex + 1) % items.length;
32            --count;
33
34            notFull.signal(); // 只唤醒生产者
35            return x;
36        } finally {
37            lock.unlock();
38        }
39    }
40 }
```

 Java

```
36         } finally {  
37             lock.unlock();  
38         }  
39     }  
40 }
```

NOTE

Condition 比 wait/notify 更灵活，适合复杂的线程协作场景。

1.3 BlockingQueue

BlockingQueue 是线程安全的队列，内置阻塞功能。

常用实现

实现类	特点	适用场景
ArrayBlockingQueue	有界，数组实现	已知容量上限
LinkedBlockingQueue	可选有界，链表实现	吞吐量要求高
PriorityBlockingQueue	无界，优先级队列	需要优先级排序
DelayQueue	无界，延迟队列	定时任务
SynchronousQueue	容量为 0，直接传递	线程池

使用示例

```
1  import java.util.concurrent.*;  
2  
3  public class BlockingQueueExample {  
4      private final BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(10);  
5  
6      public void produce() throws InterruptedException {  
7          int value = 0;  
8          while (true) {  
9              queue.put(value); // 队列满时阻塞  
10             System.out.println("Produced: " + value++);  
11             Thread.sleep(100);  
12         }  
13     }  
14  
15     public void consume() throws InterruptedException {  
16         while (true) {  
17             Integer value = queue.take(); // 队列空时阻塞  
18             System.out.println("Consumed: " + value);  
19             Thread.sleep(150);  
20         }  
21     }  
22 }
```

阻塞方法：

- `put()`: 队列满时阻塞
- `take()`: 队列空时阻塞
- `offer(e, time, unit)`: 超时阻塞
- `poll(time, unit)`: 超时阻塞

TIP

BlockingQueue 是实现生产者-消费者模式的最简单方式，推荐优先使用。

2 线程池

线程池是管理和复用线程的框架，避免频繁创建销毁线程的开销。

2.1 为什么需要线程池

问题：

```
1 // 不好的做法：每次请求都创建新线程
2 for (int i = 0; i < 1000; i++) {
3     new Thread(() -> {
4         // 处理任务
5     }).start();
6 }
```

 Java

缺点：

- 频繁创建销毁线程，性能差
- 无法控制线程数量，可能导致 OOM
- 缺乏统一管理，难以监控

线程池的优势：

- 降低资源消耗（复用线程）
- 提高响应速度（无需创建线程）
- 提高线程可管理性（统一调度）

2.2 ThreadPoolExecutor 核心参数

```
1 public ThreadPoolExecutor(
2     int corePoolSize,           // 核心线程数
3     int maximumPoolSize,       // 最大线程数
4     long keepAliveTime,        // 空闲线程存活时间
5     TimeUnit unit,             // 时间单位
6     BlockingQueue<Runnable> workQueue, // 工作队列
7     ThreadFactory threadFactory, // 线程工厂
8     RejectedExecutionHandler handler // 拒绝策略
9 )
```

 Java

参数详解

corePoolSize（核心线程数）

- 线程池中保持的最小线程数
- 即使空闲也不会被回收（除非 `allowCoreThreadTimeOut`）
- 新任务到达时，如果当前线程数 $< \text{corePoolSize}$ ，创建新线程

maximumPoolSize（最大线程数）

- 线程池中允许的最大线程数
- 当队列满且当前线程数 $< \text{maximumPoolSize}$ 时，创建非核心线程

keepAliveTime（空闲线程存活时间）

- 非核心线程的空闲超时时间
- 超过这个时间的空闲非核心线程会被回收

workQueue（工作队列）

存储待执行任务的队列：

队列类型	特点	风险
<code>ArrayBlockingQueue</code>	有界队列	可能触发拒绝策略
<code>LinkedBlockingQueue</code>	无界队列（默认 <code>Integer.MAX_VALUE</code> ）	可能 OOM
<code>SynchronousQueue</code>	不存储，直接传递	需要较大的 <code>maximumPoolSize</code>
<code>PriorityBlockingQueue</code>	优先级队列	可能 OOM

threadFactory（线程工厂）

用于创建线程，可以自定义线程名称、优先级等：

```
1 ThreadFactory factory = new ThreadFactory() {  
2     private final AtomicInteger count = new AtomicInteger(1);  
3  
4     @Override  
5     public Thread newThread(Runnable r) {  
6         Thread thread = new Thread(r);  
7         thread.setName("my-pool-" + count.getAndIncrement());  
8         thread.setDaemon(false);  
9         return thread;  
10    }  
11 };
```

handler（拒绝策略）

当队列满且达到最大线程数时的处理策略：

策略	行为	适用场景
<code>AbortPolicy</code>	抛出 <code>RejectedExecutionException</code>	默认，重要任务
<code>CallerRunsPolicy</code>	由调用线程执行	不允许丢失任务
<code>DiscardPolicy</code>	静默丢弃	允许丢失

DiscardOldestPolicy

丢弃最老的任务

允许丢失旧任务

2.3 线程池工作流程

```
1  新任务提交
2  ↓
3  当前线程数 < corePoolSize?
4  ↳ 是: 创建核心线程执行
5  ↳ 否: ↓
6  ↓
7  工作队列已满?
8  ↳ 否: 加入工作队列等待
9  ↳ 是: ↓
10 ↓
11 当前线程数 < maximumPoolSize?
12 ↳ 是: 创建非核心线程执行
13 ↳ 否: ↓
14 ↓
15 执行拒绝策略
```

TIP

理解这个流程是掌握线程池的关键!

2.4 线程池的使用

手动创建 (推荐)

```
1  ThreadPoolExecutor executor = new ThreadPoolExecutor(
2      5, // 核心线程数
3      10, // 最大线程数
4      60L, TimeUnit.SECONDS, // 空闲线程存活60秒
5      new ArrayBlockingQueue<>(100), // 有界队列
6      new ThreadPoolExecutor.CallerRunsPolicy() // 拒绝策略
7  );
8
9  // 提交任务
10 executor.submit(() -> {
11     System.out.println("Task executed by: " + Thread.currentThread().getName());
12 });
13
14 // 关闭线程池
15 executor.shutdown(); // 优雅关闭
16 // executor.shutdownNow(); // 立即关闭
```

 Java

Executors 工厂方法 (不推荐)

```
1  // 固定大小线程池
2  ExecutorService fixedPool = Executors.newFixedThreadPool(10);
3
4  // 单线程池
```

 Java

```
5  ExecutorService singlePool = Executors.newSingleThreadExecutor();
6
7  // 缓存线程池（无界，危险！）
8  ExecutorService cachedPool = Executors.newCachedThreadPool();
9
10 // 定时任务线程池
11 ScheduledExecutorService scheduledPool = Executors.newScheduledThreadPool(5);
```

CAUTION

《阿里巴巴 Java 开发手册》禁止使用 Executors 创建线程池，因为：

- FixedThreadPool 和 SingleThreadPool：队列无界，可能 OOM
- CachedThreadPool：线程数无界，可能 OOM
- 应该手动创建 ThreadPoolExecutor

2.5 线程池监控

```
1  ThreadPoolExecutor executor = new ThreadPoolExecutor(...);
2
3  // 监控指标
4  System.out.println("活跃线程数: " + executor.getActiveCount());
5  System.out.println("已完成任务数: " + executor.getCompletedTaskCount());
6  System.out.println("当前线程数: " + executor.getPoolSize());
7  System.out.println("核心线程数: " + executor.getCorePoolSize());
8  System.out.println("最大线程数: " + executor.getMaximumPoolSize());
9  System.out.println("队列大小: " + executor.getQueue().size());
```

 Java

2.6 线程池最佳实践

合理设置参数

CPU 密集型任务：

```
1  int corePoolSize = Runtime.getRuntime().availableProcessors() + 1;
```

 Java

IO 密集型任务：

```
1  int corePoolSize = Runtime.getRuntime().availableProcessors() * 2;
2  // 或
3  double corePoolSize = Runtime.getRuntime().availableProcessors() / (1 - 阻塞系数);
4  // 阻塞系数通常在0.8~0.9之间
```

 Java

自定义线程名称

```
1  ThreadFactory factory = new ThreadFactoryBuilder()
2      .setNameFormat("order-pool-%d")
3      .setDaemon(false)
4      .build();
5
6  ThreadPoolExecutor executor = new ThreadPoolExecutor(
7      5, 10, 60L, TimeUnit.SECONDS,
8      new ArrayBlockingQueue<>(100),
9      factory
```

 Java

```
10 );
```

TIP

自定义线程名称便于问题排查，特别是在生产环境中。

优雅关闭

```
1 public void shutdownGracefully(ThreadPoolExecutor executor) {  
2     executor.shutdown(); // 不再接受新任务  
3     try {  
4         if (!executor.awaitTermination(60, TimeUnit.SECONDS)) {  
5             executor.shutdownNow(); // 强制关闭  
6             if (!executor.awaitTermination(60, TimeUnit.SECONDS)) {  
7                 System.err.println("线程池未能正常关闭");  
8             }  
9         }  
10    } catch (InterruptedException e) {  
11        executor.shutdownNow();  
12        Thread.currentThread().interrupt();  
13    }  
14 }
```

异常处理

```
1 // submit方式的异常不会直接抛出  
2 Future<?> future = executor.submit(() -> {  
3     throw new RuntimeException("Task failed");  
4 });  
5  
6 try {  
7     future.get(); // 这里才会抛出异常  
8 } catch (ExecutionException e) {  
9     e.getCause().printStackTrace();  
10 }  
11  
12 // execute方式的异常需要通过UncaughtExceptionHandler处理  
13 executor.setThreadFactory(r -> {  
14     Thread t = new Thread(r);  
15     t.setUncaughtExceptionHandler((thread, ex) -> {  
16         System.err.println("Thread " + thread.getName() + " failed: " + ex.getMessage());  
17     });  
18     return t;  
19 });
```

3 JUC 工具类

java.util.concurrent 包提供了丰富的并发工具类。

3.1 CountdownLatch

允许一个或多个线程等待其他线程完成操作。 ...

```
1 public class CountdownLatchExample {
2     public static void main(String[] args) throws InterruptedException {
3         int threadCount = 5;
4         CountdownLatch latch = new CountdownLatch(threadCount);
5
6         for (int i = 0; i < threadCount; i++) {
7             new Thread(() -> {
8                 System.out.println(Thread.currentThread().getName() + " working...");
9                 try {
10                     Thread.sleep(1000);
11                 } catch (InterruptedException e) {
12                     e.printStackTrace();
13                 }
14                 System.out.println(Thread.currentThread().getName() + " done");
15                 latch.countDown(); // 计数减1
16             }, "Worker-" + i).start();
17         }
18
19         latch.await(); // 等待所有线程完成
20         System.out.println("All workers completed!");
21     }
22 }
```

特点:

- 计数器只能递减，不能重置
- 一次性使用
- 适合等待多个并行任务完成

3.2 CyclicBarrier

让一组线程互相等待，直到所有线程都到达某个屏障点。

```
1 public class CyclicBarrierExample {
2     public static void main(String[] args) {
3         int threadCount = 3;
4         CyclicBarrier barrier = new CyclicBarrier(threadCount, () -> {
5             System.out.println("All threads reached barrier, proceeding...");
6         });
7
8         for (int i = 0; i < threadCount; i++) {
9             new Thread(() -> {
10                 try {
11                     System.out.println(Thread.currentThread().getName() + " preparing...");
12                     Thread.sleep((long) (Math.random() * 3000));
13
14                     System.out.println(Thread.currentThread().getName() + " waiting at barrier");
15                     barrier.await(); // 等待其他线程
16
17                     System.out.println(Thread.currentThread().getName() + " continuing...");
18                 } catch (Exception e) {
```

```
19         e.printStackTrace();
20     }
21     }, "Thread-" + i).start();
22 }
23 }
24 }
```

特点:

- 可以重复使用 (reset)
- 所有线程同时继续执行
- 适合分阶段并行计算

NOTE

CountDownLatch 是一个线程等待其他线程, CyclicBarrier 是一组线程互相等待。

3.3 Semaphore

控制同时访问某个资源的线程数量。

```
1 public class SemaphoreExample {
2     private final Semaphore semaphore = new Semaphore(3); // 最多3个并发
3
4     public void accessResource() {
5         try {
6             semaphore.acquire(); // 获取许可
7             System.out.println(Thread.currentThread().getName() + " accessing resource");
8             Thread.sleep(2000);
9             System.out.println(Thread.currentThread().getName() + " releasing resource");
10        } catch (InterruptedException e) {
11            e.printStackTrace();
12        } finally {
13            semaphore.release(); // 释放许可
14        }
15    }
16 }
```

应用场景:

- 限流
- 控制数据库连接数
- 控制并发访问数

3.4 CompletableFuture

异步编程的强大工具, 支持链式调用和组合。

基本用法

```
1 import java.util.concurrent.*;
2
3 public class CompletableFutureExample {
4     public static void main(String[] args) throws Exception {
```

```
5      // 异步执行
6      CompletableFuture<String> future = CompletableFuture.supplyAsync(() -> {
7          System.out.println("Executing in: " + Thread.currentThread().getName());
8          return "Hello";
9      });
10
11     // 获取结果
12     String result = future.get();
13     System.out.println("Result: " + result);
14 }
15 }
```

链式调用

```
1 CompletableFuture<String> future = CompletableFuture
2     .supplyAsync(() -> "Hello")
3     .thenApply(s -> s + " World")
4     .thenApply(String::toUpperCase)
5     .thenAccept(System.out::println); // HELLO WORLD
6
7 future.join(); // 等待完成
```

 Java

组合多个 CompletableFuture

```
1 CompletableFuture<String> future1 = CompletableFuture.supplyAsync(() -> "Hello");
2 CompletableFuture<String> future2 = CompletableFuture.supplyAsync(() -> "World");
3
4 // thenCombine: 两个都完成后执行
5 CompletableFuture<String> combined = future1.thenCombine(future2, (s1, s2) -> s1 + " " + s2);
6 System.out.println(combined.join()); // Hello World
7
8 // allOf: 所有完成后执行
9 CompletableFuture<Void> all = CompletableFuture.allOf(future1, future2);
10 all.join();
11
12 // anyOf: 任意一个完成后执行
13 CompletableFuture<Object> any = CompletableFuture.anyOf(future1, future2);
14 System.out.println(any.join());
```

 Java

异常处理

```
1 CompletableFuture<String> future = CompletableFuture
2     .supplyAsync(() -> {
3         if (Math.random() > 0.5) {
4             throw new RuntimeException("Error!");
5         }
6         return "Success";
7     })
8     .exceptionally(ex -> {
9         System.err.println("Exception: " + ex.getMessage());
10        return "Fallback";
11    })
```

 Java


```
12     .handle((result, ex) -> {
13         if (ex != null) {
14             return "Handled: " + ex.getMessage();
15         }
16         return result;
17     });
18
19 System.out.println(future.join());
```

TIP

CompletableFuture 是现代 Java 异步编程的首选，比传统的 Future 更强大和灵活。

3.5 Exchanger

用于两个线程之间交换数据。

```
1 public class ExchangerExample {
2     public static void main(String[] args) {
3         Exchanger<String> exchanger = new Exchanger<>();
4
5         new Thread(() -> {
6             try {
7                 String data = "Data from Thread 1";
8                 System.out.println("Thread 1 sending: " + data);
9                 String received = exchanger.exchange(data);
10                System.out.println("Thread 1 received: " + received);
11            } catch (InterruptedException e) {
12                e.printStackTrace();
13            }
14        }).start();
15
16        new Thread(() -> {
17            try {
18                String data = "Data from Thread 2";
19                System.out.println("Thread 2 sending: " + data);
20                String received = exchanger.exchange(data);
21                System.out.println("Thread 2 received: " + received);
22            } catch (InterruptedException e) {
23                e.printStackTrace();
24            }
25        }).start();
26    }
27 }
```

4 总结

并发协作与线程池的核心要点：

- 线程通信：wait/notify、Condition、BlockingQueue
- 线程池：ThreadPoolExecutor 七大参数、工作流程、最佳实践
- JUC 工具：

- CountdownLatch: 一个等待多个
- CyclicBarrier: 多个互相等待
- Semaphore: 控制并发数
- CompletableFuture: 异步编程
- Exchanger: 线程间交换数据

...

下一章将探讨并发容器与原子类的使用。

Chapter 7

Kotlin 协程基础

Kotlin 协程是一种轻量级的并发框架，通过挂起和恢复机制实现高效的异步编程。它是 Kotlin 语言的核心特性之一。

NOTE

协程的核心理念是用户态线程，由程序员控制调度，而非操作系统，因此更加轻量 and 高效。

1 协程模型

1.1 什么是协程

协程（Coroutine）是一种用户态的轻量级线程，可以在执行过程中暂停和恢复。

特点：

- 轻量级：一个线程可以运行成千上万个协程
- 协作式：协程主动让出执行权，而非被抢占
- 结构化：有明确的生命周期和作用域

```
1  import kotlinx.coroutines.*
2
3  fun main() = runBlocking {
4      // 启动一个协程
5      launch {
6          delay(1000L) // 非阻塞延迟
7          println("World!")
8      }
9      println("Hello,")
10 }
11 // 输出：
12 // Hello,
13 // World!
```

1.2 协程 vs 线程

特性	线程	协程
创建开销	高（MB 级栈空间）	低（KB 级）
切换成本	高（内核态切换）	低（用户态切换）
数量限制	几百到几千	数万到数百万
调度方式	操作系统调度	程序员控制

阻塞	阻塞线程	挂起不阻塞线程
内存占用	~1MB/线程	~几 KB/协程

示例对比：

```
1 // 线程方式：创建10000个线程会导致OOM
2 for (i in 1..10000) {
3     Thread {
4         Thread.sleep(1000)
5         println("Thread $i")
6     }.start()
7 }
8
9 // 协程方式：轻松创建10000个协程
10 runBlocking {
11     for (i in 1..10000) {
12         launch {
13             delay(1000)
14             println("Coroutine $i")
15         }
16     }
17 }
```

TIP

协程不是替代线程，而是建立在线程之上的更高层抽象。

1.3 协程的优势

1. 高性能：

- 轻量级，可以创建大量协程
- 无锁设计，减少竞争

2. 易读性：

- 异步代码像同步代码一样书写
- 避免回调地狱

3. 结构化并发：

- 自动管理生命周期
- 异常传播清晰
- 资源自动清理

4. 灵活性：

- 可以轻松切换执行上下文
- 支持取消和超时

1.4 结构化并发

结构化并发确保协程的生命周期清晰可控。

原则：

1. 子协程的作用域不能超过父协程
2. 父协程等待所有子协程完成

3. 异常向上传播

```
1 fun main() = runBlocking {  
2     // 父协程  
3     launch {  
4         // 子协程1  
5         launch {  
6             delay(1000)  
7             println("Child 1")  
8         }  
9  
10        // 子协程2  
11        launch {  
12            delay(500)  
13            println("Child 2")  
14        }  
15  
16        println("Parent waiting...")  
17    } // 父协程会等待所有子协程完成  
18  
19    println("All done")  
20 }
```

优势:

- 防止协程泄漏
- 自动取消子协程
- 清晰的错误处理

CAUTION

避免使用 GlobalScope，它破坏了结构化并发原则。

2 核心概念

2.1 挂起函数 (Suspend Function)

挂起函数是可以暂停执行而不阻塞线程的函数。

定义挂起函数

```
1 import kotlinx.coroutines.*  
2  
3 // 普通函数不能调用挂起函数  
4 // suspend函数可以调用其他suspend函数  
5 suspend fun fetchData(): String {  
6     delay(1000) // 挂起点，不阻塞线程  
7     return "Data"  
8 }  
9  
10 suspend fun processData(): String {  
11     val data = fetchData() // 调用挂起函数
```

```
12     return data.uppercase()
13 }
```

关键字: `suspend`

特点:

- 只能在协程或其他挂起函数中调用
- 可以在不阻塞线程的情况下暂停执行
- 编译器将其转换为状态机

挂起函数的原理

```
1 普通函数调用:
2 调用 → 执行 → 返回
3
4 挂起函数调用:
5 调用 → 执行 → 挂起 (保存状态) → 恢复 (从保存点继续) → 返回
```

编译器转换:

```
1 // 原始代码
2 suspend fun example() {
3     println("Before")
4     delay(1000) // 挂起点
5     println("After")
6 }
7
8 // 编译器生成的伪代码 (简化)
9 fun example(continuation: Continuation) {
10     when (continuation.label) {
11         0 -> {
12             println("Before")
13             continuation.label = 1
14             delay(1000, continuation) // 传入continuation
15             return
16         }
17         1 -> {
18             println("After")
19             continuation.resume(Unit)
20         }
21     }
22 }
```

NOTE

挂起函数不是新的线程，而是在现有线程上通过状态机实现的协作式多任务。

2.2 协程构建器

launch

启动一个新协程，返回 Job 对象。

```
1 fun main() = runBlocking {  
2     val job = launch {  
3         delay(1000)  
4         println("Launch coroutine")  
5     }  
6  
7     println("Main continues...")  
8     job.join() // 等待协程完成  
9     println("Done")  
10 }
```

特点:

- 返回 `Job`
- 用于“发射并忘记”的场景
- 异常会传递给父协程

| async

启动一个新协程，返回 `Deferred` 对象（继承自 `Job`）。

```
1 fun main() = runBlocking {  
2     val deferred = async {  
3         delay(1000)  
4         "Result"  
5     }  
6  
7     println("Main continues...")  
8     val result = deferred.await() // 获取结果  
9     println("Result: $result")  
10 }
```

特点:

- 返回 `Deferred<T>`
- 用于需要返回值的场景
- `await()` 获取结果

| 对比

```
1 // launch: 不需要返回值  
2 launch {  
3     doSomeWork()  
4 }  
5  
6 // async: 需要返回值  
7 val result = async {  
8     computeSomething()  
9 }.await()
```

TIP

如果不需要返回值，使用 `launch`；如果需要返回值，使用 `async`。

2.3 协程作用域 (CoroutineScope)

作用域定义了协程的生命周期和上下文。

I 常用作用域

runBlocking

阻塞当前线程，直到所有协程完成。

```
1 fun main() = runBlocking {  
2     // 这个协程会阻塞main线程  
3     launch {  
4         delay(1000)  
5         println("Done")  
6     }  
7 }
```

用途：

- 测试代码
- main 函数入口
- 桥接阻塞和非阻塞代码

coroutineScope

创建一个新作用域，等待所有子协程完成。

```
1 suspend fun doConcurrentWork() = coroutineScope {  
2     val task1 = async { fetchData1() }  
3     val task2 = async { fetchData2() }  
4  
5     // 并行执行  
6     val result1 = task1.await()  
7     val result2 = task2.await()  
8  
9     process(result1, result2)  
10 } // 这里会等待所有子协程完成
```

特点：

- 挂起函数
- 等待所有子协程
- 异常传播

GlobalScope (不推荐)

全局作用域，生命周期与应用相同。

```
1 // 不推荐!  
2 GlobalScope.launch {  
3     delay(1000)  
4     println("This might leak!")  
5 }
```


CAUTION

避免使用 `GlobalScope`，它会导致协程泄漏，破坏结构化并发。

MainScope / lifecycleScope (Android)

```
1 // Android ViewModel
2 class MyViewModel : ViewModel() {
3     val viewModelScope = viewModelScope // 自动绑定ViewModel生命周期
4
5     fun loadData() {
6         viewModelScope.launch {
7             // ViewModel销毁时自动取消
8         }
9     }
10 }
11
12 // Android Activity/Fragment
13 class MyActivity : AppCompatActivity() {
14     override fun onCreate(savedInstanceState: Bundle?) {
15         super.onCreate(savedInstanceState)
16
17         lifecycleScope.launch {
18             // Activity销毁时自动取消
19         }
20     }
21 }
```

自定义作用域

```
1 class MyClass {
2     private val scope = CoroutineScope(SupervisorJob() + Dispatchers.Default)
3
4     fun doWork() {
5         scope.launch {
6             // 工作
7         }
8     }
9
10    fun cleanup() {
11        scope.cancel() // 取消所有协程
12    }
13 }
```

2.4 取消与超时

协程取消

```
1 fun main() = runBlocking {
2     val job = launch {
3         repeat(1000) { i ->
4             println("Working... $i")
5             delay(100)
6         }
7     }
8 }
```

```
6      }
7    }
8
9    delay(250)
10   println("Cancelling...")
11   job.cancel() // 取消协程
12   job.join()   // 等待取消完成
13   println("Cancelled!")
14 }
```

取消是协作式的：

```
1 val job = launch {
2     while (isActive) { // 检查是否被取消
3         // 做工作
4         if (isCancelled) break
5     }
6 }
```

不可取消的操作：

```
1 val job = launch {
2     try {
3         // 这段代码不会被中断
4         doCriticalWork()
5     } finally {
6         // 清理资源（即使被取消也会执行）
7         cleanup()
8     }
9 }
10 job.cancel()
```

TIP

使用 `withContext(NonCancellable)` 来执行不可取消的操作。

| 超时处理

```
1 fun main() = runBlocking {
2     try {
3         withTimeout(1000) {
4             repeat(1000) { i ->
5                 println("Working... $i")
6                 delay(200)
7             }
8         }
9     } catch (e: TimeoutCancellationException) {
10         println("Timed out!")
11     }
12 }
```

不抛出异常的版本：

```
1 val result = withTimeoutOrNull(1000) {
2     // 长时间运行的操作
```

```
3     delay(2000)
4     "Result"
5 }
6
7 if (result == null) {
8     println("Timed out")
9 } else {
10    println("Result: $result")
11 }
```

3 协程上下文与调度器

协程上下文定义了协程的执行环境。

3.1 协程上下文组成

```
1 CoroutineContext = Job + CoroutineDispatcher + CoroutineName + ...
```



主要元素：

元素	作用	示例
Job	生命周期管理	cancel(), join()
Dispatcher	线程调度	Dispatchers.IO
CoroutineName	调试名称	CoroutineName("MyCoroutine")
ExceptionHandler	异常处理	CoroutineExceptionHandler

3.2 调度器 (Dispatcher)

调度器决定协程在哪个线程上执行。

主要调度器

Dispatchers.Default

用途：CPU 密集型任务

线程池：大小等于 CPU 核心数

```
1 launch(Dispatchers.Default) {
2     // CPU密集型计算
3     val result = heavyComputation()
4 }
```



Dispatchers.IO

用途：IO 密集型任务（网络、文件、数据库）

线程池：默认 64 个线程或 CPU 核心数（取较大值）

```
1 launch(Dispatchers.IO) {  
2     // IO操作  
3     val data = readFile()  
4     val response = httpClient.get(url)  
5 }
```

Kotlin

Dispatchers.Main

用途：Android 主线程（UI 更新）

依赖：需要添加 `kotlinx-coroutines-android`

```
1 // Android  
2 launch(Dispatchers.Main) {  
3     // 更新UI  
4     textView.text = "Updated"  
5 }
```

Kotlin

Dispatchers.Unconfined

用途：不限制线程，在当前线程开始，在恢复的线程继续

```
1 launch(Dispatchers.Unconfined) {  
2     println("Start: ${Thread.currentThread().name}")  
3     delay(100)  
4     println("After delay: ${Thread.currentThread().name}") // 可能不同线程  
5 }
```

Kotlin

CAUTION

Dispatchers.Unconfined 主要用于测试，生产环境慎用。

切换调度器

```
1 fun main() = runBlocking {  
2     println("Main: ${Thread.currentThread().name}")  
3  
4     withContext(Dispatchers.Default) {  
5         println("Default: ${Thread.currentThread().name}")  
6  
7         withContext(Dispatchers.IO) {  
8             println("IO: ${Thread.currentThread().name}")  
9         }  
10  
11         println("Back to Default: ${Thread.currentThread().name}")  
12     }  
13  
14     println("Back to Main: ${Thread.currentThread().name}")  
15 }
```

Kotlin

最佳实践：

```
1 suspend fun loadAndProcessData() {  
2     // IO线程加载数据
```

Kotlin

```
3     val data = withContext(Dispatchers.IO) {
4         database.query()
5     }
6
7     // Default线程处理数据
8     val result = withContext(Dispatchers.Default) {
9         data.map { transform(it) }
10    }
11
12    // 返回结果（在调用者的上下文中）
13    return result
14 }
```

TIP

遵循“在哪里使用，就在哪里切换”的原则，让调用者决定最终的执行上下文。

3.3 上下文继承

子协程继承父协程的上下文。

```
1 fun main() = runBlocking(CoroutineName("Parent")) {
2     println("Parent: ${coroutineContext[CoroutineName]}")
3
4     launch { // 继承父的CoroutineName
5         println("Child: ${coroutineContext[CoroutineName]}")
6     }
7
8     launch(CoroutineName("CustomChild")) { // 覆盖CoroutineName
9         println("Custom Child: ${coroutineContext[CoroutineName]}")
10    }
11 }
```

 Kotlin

3.4 自定义上下文元素

```
1 // 自定义日志上下文
2 data class LoggerContext(val tag: String) : CoroutineContext.Element {
3     companion object Key : CoroutineContext.Key<LoggerContext>
4 }
5
6 fun main() = runBlocking {
7     val scope = CoroutineScope(LoggerContext("MyTag") + Dispatchers.Default)
8
9     scope.launch {
10         val logger = coroutineContext[LoggerContext]
11         println("[${logger?.tag}] Working...")
12     }
13 }
```

 Kotlin

3.5 异常处理

CoroutineExceptionHandler

```
1 val handler = CoroutineExceptionHandler { context, exception ->
2     println("Caught exception: ${exception.message}")
3 }
4
5 val scope = CoroutineScope(Job() + handler)
6
7 scope.launch {
8     throw RuntimeException("Error!")
9 }
```

注意:

- 只对顶层协程有效
- launch 的异常会传递给父协程
- async 的异常在 await()时抛出

SupervisorJob

允许子协程失败而不影响其他子协程。

```
1 val scope = CoroutineScope(SupervisorJob())
2
3 scope.launch {
4     throw RuntimeException("This won't cancel siblings")
5 }
6
7 scope.launch {
8     delay(1000)
9     println("This will still execute")
10 }
```

vs Job:

特性	Job	SupervisorJob
子协程失败	取消所有子协程	只取消失败的
异常传播	向上传播	需要 Handler 捕获
适用场景	强关联任务	独立任务

4 实战示例

4.1 并行数据加载

```
1 suspend fun loadDashboardData(): DashboardData {
2     return coroutineScope {
3         // 并行加载
4         val userDeferred = async { fetchUserProfile() }
5         val postsDeferred = async { fetchUserPosts() }
6         val statsDeferred = async { fetchUserStats() }
```

```
7
8      // 等待所有结果
9      DashboardData(
10         user = userDeferred.await(),
11         posts = postsDeferred.await(),
12         stats = statsDeferred.await()
13     )
14 }
15 }
```

4.2 重试机制

```
1 suspend fun <T> retry(
2     times: Int = 3,
3     initialDelay: Long = 100,
4     maxDelay: Long = 1000,
5     factor: Double = 2.0,
6     block: suspend () -> T
7 ): T {
8     var currentDelay = initialDelay
9     repeat(times - 1) {
10         try {
11             return block()
12         } catch (e: Exception) {
13             println("Retry ${it + 1}: ${e.message}")
14         }
15         delay(currentDelay)
16         currentDelay = (currentDelay * factor).toLong().coerceAtMost(maxDelay)
17     }
18     return block() // 最后一次尝试
19 }
20
21 // 使用
22 val data = retry {
23     apiService.fetchData()
24 }
```

4.3 Flow 集成

```
1 import kotlinx.coroutines.flow.*
2
3 fun observeData(): Flow<String> = flow {
4     for (i in 1..10) {
5         delay(100)
6         emit("Data $i")
7     }
8 }
9
10 fun main() = runBlocking {
11     observeData()
12         .filter { it.contains("5") }
```

```
13         .map { it.uppercase() }  
14         .collect { println(it) }  
15     }
```

5 总结

Kotlin 协程的核心要点：

- 协程模型：轻量级、用户态、结构化并发
- 挂起函数：suspend 关键字，状态机实现
- 构建器：launch（无返回值）、async（有返回值）
- 作用域：runBlocking、coroutineScope、避免 GlobalScope
- 取消与超时：协作式取消、withTimeout
- 调度器：Default（CPU）、IO（IO）、Main（UI）
- 上下文：Job + Dispatcher + 其他元素
- 异常处理：CoroutineExceptionHandler、SupervisorJob

下一章将探讨并发容器与原子类的使用。

Chapter 8

协程进阶与异步流

Chapter 9

并发诊断与综合实战



网络编程与高性能 IO



JVM 底层原理与性能调优

Chapter 10

GraalVM

GraalVM 是一个高性能的通用虚拟机，由 Oracle 主导开发。它不仅能运行 Java 字节码，还能运行 JavaScript、Python、Ruby、R 等多种语言，并提供了原生镜像（Native Image）编译能力，显著提升了启动速度和降低了内存占用。

NOTE

GraalVM 的核心理念是多语言支持和高性能，通过 Ahead-of-Time (AOT) 编译技术，将 Java 应用编译为原生可执行文件，实现毫秒级启动和极低的内存占用。

1 GraalVM 概述

1.1 发展历程

- 2018 年：Oracle 正式发布 GraalVM
- 2019 年：推出 GraalVM Enterprise Edition
- 2020 年：GraalVM 成为 OpenJDK 项目
- 2021 年：Spring Boot 2.4 开始支持 GraalVM 原生镜像
- 2022 年：Spring Boot 3.0 全面支持 GraalVM
- 现在：云原生 Java 应用的首选运行时

1.2 核心组件

▮ Graal 编译器

- 高性能的 JIT 编译器
- 用 Java 编写，易于扩展
- 支持多种语言
- 比 C2 编译器性能更好

▮ Native Image

- AOT 编译工具
- 将 Java 应用编译为原生可执行文件
- 无需 JVM 即可运行
- 启动速度快，内存占用低

▮ Polyglot API

- 多语言互操作性
- 在 Java 中调用其他语言代码

- 共享数据和对象
- 零拷贝性能

Truffle 框架

- 语言实现框架
- 简化新语言的实现
- 自动获得 Graal 编译器优化

1.3 GraalVM vs 传统 JVM

特性	传统 JVM	GraalVM
启动速度	秒级	毫秒级
内存占用	较高	极低
峰值性能	优秀	优秀或更好
多语言支持	有限	原生支持
容器友好性	一般	优秀
冷启动	慢	快
适用场景	长期运行服务	Serverless、微服务

TIP

GraalVM 不是要取代传统 JVM，而是在特定场景下提供更好的选择。对于长期运行的服务，传统 JVM 的 JIT 优化可能更有优势；而对于需要快速启动的场景，GraalVM 的原生镜像是更好的选择。

2 Native Image 详解

Native Image 是 GraalVM 最引人注目的特性，它将 Java 应用提前编译为原生机器码。

2.1 工作原理

构建时初始化

1 传统JVM:	Native Image:	text
2 源代码 -> 字节码	源代码 -> 字节码	
3 ↓	↓	
4 运行时JIT编译	构建时AOT编译	
5 ↓	↓	
6 机器码 (热点代码)	原生可执行文件	
7 ↓	↓	
8 运行	直接运行	

关键区别：

- 传统 JVM：运行时动态编译热点代码
- Native Image：构建时静态编译所有可达代码

封闭世界假设

Native Image 基于封闭世界假设（Closed World Assumption）：

- 构建时确定所有可达的代码和数据
- 运行时不能动态加载新的类
- 反射、JNI 等需要在构建时配置

CAUTION

封闭世界假设是 Native Image 的主要限制。任何运行时动态行为（反射、动态代理、资源加载等）都需要显式配置。

静态分析

1	入口点（main方法）	text
2	↓	
3	静态分析可达代码	
4	↓	
5	移除未使用的代码（Dead Code Elimination）	
6	↓	
7	内联优化	
8	↓	
9	生成原生机器码	
10	↓	
11	链接系统库	
12	↓	
13	生成可执行文件	

2.2 构建 Native Image

使用 native-image 命令

1	# 基本用法	Bash
2	<code>native-image -jar myapp.jar</code>	
3		
4	# 指定输出名称	
5	<code>native-image -jar myapp.jar -o myapp</code>	
6		
7	# 启用所有安全提示	
8	<code>native-image -jar myapp.jar --enable-all-security-services</code>	
9		
10	# 指定主类	
11	<code>native-image -cp myapp.jar com.example.Main</code>	

使用 Maven 插件

1	<code><plugin></code>	xml
2	<code><groupId>org.graalvm.buildtools</groupId></code>	
3	<code><artifactId>native-maven-plugin</artifactId></code>	
4	<code><version>0.9.28</version></code>	
5	<code><executions></code>	
6	<code><execution></code>	

```
7         <id>build-native</id>
8         <goals>
9             <goal>compile-no-fork</goal>
10        </goals>
11        <phase>package</phase>
12    </execution>
13 </executions>
14 </plugin>
```

```
1 # 构建原生镜像
2 mvn -Pnative native:compile
3
4 # 运行测试
5 mvn -Pnative test
```

 Bash

使用 Gradle 插件

```
1 plugins {
2     id 'org.graalvm.buildtools.native' version '0.9.28'
3 }
4
5 graalvmNative {
6     binaries {
7         main {
8             imageName.set('myapp')
9             mainClass.set('com.example.Main')
10        }
11    }
12 }
```

 groovy

```
1 # 构建原生镜像
2 ./gradlew nativeCompile
3
4 # 运行
5 ./build/native/nativeCompile/myapp
```

 Bash

TIP

Spring Boot 3.0+内置了 GraalVM 支持，使用 `spring-boot-maven-plugin` 即可轻松构建原生镜像。

2.3 性能对比

启动时间

```
1 传统JVM (Spring Boot):
2   - 启动时间: 5-10秒
3   - 首次请求: 可能有JIT编译延迟
4
5  Native Image (Spring Boot):
6   - 启动时间: 50-200毫秒
7   - 首次请求: 无额外延迟
```

 text

提升: 50-100 倍

内存占用

1	传统JVM:	text
2	- RSS内存: 200-500 MB	
3	- 堆内存: 根据配置	
4		
5	Native Image:	
6	- RSS内存: 20-50 MB	
7	- 无独立堆 (使用系统内存)	

降低: 80-90%

吞吐量

1	短期运行 (< 1分钟):
2	- Native Image: 更优 (无JIT预热)
3	
4	长期运行 (> 10分钟):
5	- 传统JVM: 可能更优 (JIT优化热点代码)
6	- Native Image: 稳定 (已优化)

NOTE

Native Image 的优势在于启动速度和内存占用, 而不是峰值吞吐量。对于长期运行的高负载服务, 传统 JVM 的 JIT 优化可能达到更高的峰值性能。

3 配置与优化

Native Image 需要特殊配置来处理反射、资源文件等动态特性。

3.1 反射配置

反射是 Native Image 最常见的配置需求。

问题示例

1	// 这段代码在Native Image中会失败	Java
2	class MyClass {	
3	public void doSomething() {	
4	// ...	
5	}	
6	}	
7		
8	// 运行时反射	
9	Class<?> clazz = Class.forName("com.example.MyClass");	
10	Object obj = clazz.getDeclaredConstructor().newInstance();	

原因: Native Image 在构建时不知道需要保留 `MyClass` 的反射元数据。

解决方案

方式 1: reflect-config.json

创建 `src/main/resources/META-INF/native-image/reflect-config.json` :

```
1 [
2   {
3     "name": "com.example.MyClass",
4     "allDeclaredConstructors": true,
5     "allPublicMethods": true,
6     "allDeclaredFields": true
7   }
8 ]
```

方式 2: `@RegisterForReflection` 注解

```
1 import org.graalvm.nativeimage.hosted.RegisterForReflection;
2
3 @RegisterForReflection
4 public class MyClass {
5     public void doSomething() {
6         // ...
7     }
8 }
```

方式 3: Spring Boot 自动配置

Spring Boot 3.0+会自动处理大部分反射配置:

```
1 // Spring会自动注册这些类用于反射
2 @Component
3 public class MyService {
4     // ...
5 }
```

TIP

Spring Boot 3.0+的 AOT 处理会自动生成大部分所需的配置文件, 大大简化了 Native Image 的配置工作。

3.2 资源文件配置

问题

```
1 // 读取资源文件
2 InputStream is = getClass().getResourceAsStream("/config.properties");
```

在 Native Image 中, 资源文件不会自动包含。

解决方案

创建 `src/main/resources/META-INF/native-image/resource-config.json` :

```
1 {
2   "resources": {
3     "includes": [
4       {"pattern": "\\Qconfig.properties\\E"},
5       {"pattern": "static/*.json"}
6     ]
7   }
8 }
```

或使用通配符：

```
1 {
2   "resources": {
3     "includes": [
4       {"pattern": ".*\\.properties$"},
5       {"pattern": "static/*.json"}
6     ]
7   }
8 }
```

3.3 JNI 配置

如果使用了 JNI，需要配置 `jni-config.json`：

```
1 [
2   {
3     "name": "com.example.NativeLibrary",
4     "methods": [
5       {"name": "nativeMethod", "parameterTypes": ["java.lang.String"]}
6     ]
7   }
8 ]
```

3.4 代理模式配置

Spring AOP、Hibernate 等使用动态代理，需要配置：

```
1 [
2   {
3     "interfaces": [
4       "com.example.MyInterface"
5     ]
6   }
7 ]
```

NOTE

Spring Boot 3.0+ 的 AOT 处理会自动检测并配置大部分代理场景，通常不需要手动配置。

3.5 构建时初始化

有些类需要在构建时初始化，而不是运行时：

```
1 # application.properties
2 spring.native.remove-unused-autoconfig=true
3 spring.native.remove-yaml-support=true
```

properties

或在 `native-image.properties` 中配置：

```
1 Args = --initialize-at-build-time=com.example.MyClass
```

properties

4 Spring Boot 与 GraalVM

Spring Boot 3.0+对 GraalVM 提供了原生支持，使得构建原生镜像变得非常简单。

4.1 Spring Boot 3.0 的新特性

┆ AOT 处理

Spring Boot 3.0 引入了 AOT (Ahead-of-Time) 处理引擎：

- 在构建时分析 Spring 应用上下文
- 生成 GraalVM 所需的配置文件
- 优化 Bean 定义和依赖注入

```
1 传统Spring Boot:
2   启动时扫描 -> 创建Bean -> 依赖注入 -> 运行
3
4 Spring Boot + GraalVM:
5   构建时AOT处理 -> 生成配置 -> 编译为原生镜像 -> 运行
```

text

┆ 自动配置优化

Spring Boot 会自动：

- 检测并注册反射类
- 配置资源文件
- 处理动态代理
- 优化条件注解

4.2 实践示例

┆ 创建 Spring Boot 应用

```
1 @SpringBootApplication
2 @RestController
3 public class DemoApplication {
4
5     public static void main(String[] args) {
6         SpringApplication.run(DemoApplication.class, args);
7     }
8
9     @GetMapping("/hello")
10    public String hello() {
```

Java

```
11     return "Hello from GraalVM!";
12 }
13 }
```

1 pom.xml 配置

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5         https://maven.apache.org/xsd/maven-4.0.0.xsd">
6     <modelVersion>4.0.0</modelVersion>
7
8     <parent>
9         <groupId>org.springframework.boot</groupId>
10        <artifactId>spring-boot-starter-parent</artifactId>
11        <version>3.2.0</version>
12    </parent>
13
14    <groupId>com.example</groupId>
15    <artifactId>graalvm-demo</artifactId>
16    <version>1.0.0</version>
17
18    <dependencies>
19        <dependency>
20            <groupId>org.springframework.boot</groupId>
21            <artifactId>spring-boot-starter-web</artifactId>
22        </dependency>
23    </dependencies>
24
25    <build>
26        <plugins>
27            <plugin>
28                <groupId>org.springframework.boot</groupId>
29                <artifactId>spring-boot-maven-plugin</artifactId>
30            </plugin>
31            <plugin>
32                <groupId>org.graalvm.buildtools</groupId>
33                <artifactId>native-maven-plugin</artifactId>
34            </plugin>
35        </plugins>
36    </build>
37 </project>
```

1 构建和运行

```
1 # 构建JAR
2 mvn package
3
4 # 构建原生镜像
5 mvn -Pnative native:compile
6
7 # 运行原生镜像
```

 Bash

```
8  ./target/graalvm-demo
9
10 # 测试
11 curl http://localhost:8080/hello
```

4.3 常见陷阱与解决

❶ 陷阱 1：反射问题

问题：JSON 序列化/反序列化失败

解决：

```
1  // 使用Jackson, Spring会自动配置
2  @RestController
3  public class UserController {
4
5      @PostMapping("/users")
6      public User createUser(@RequestBody User user) {
7          return user;
8      }
9  }
10
11 // Jackson会自动处理反射配置
12 public class User {
13     private String name;
14     private int age;
15
16     // 必须有getter/setter或public字段
17     public String getName() { return name; }
18     public void setName(String name) { this.name = name; }
19     public int getAge() { return age; }
20     public void setAge(int age) { this.age = age; }
21 }
```

❷ 陷阱 2：资源文件缺失

问题：找不到配置文件或静态资源

解决：

```
1 # application.properties
2 # Spring Boot会自动包含以下资源
3 spring.mvc.static-path-pattern=/static/**
4 spring.web.resources.static-locations=classpath:/static/
```

陷阱 3：第三方库兼容性

问题：某些库不支持 Native Image

解决：

```
1 <!-- 检查库是否有GraalVM支持 -->
2 <dependency>
3     <groupId>com.example</groupId>
```

```
4 <artifactId>some-library</artifactId>
5 <version>1.0.0</version>
6 <!-- 寻找有native-image配置的版本 -->
7 </dependency>
```

TIP

在选择第三方库时，优先选择已经提供 GraalVM 支持的库。可以查看库的文档或 GitHub 仓库是否有 `META-INF/native-image` 目录。

I 陷阱 4：启动参数

问题：运行时传递的参数不生效

解决：

```
1 # Native Image支持大部分JVM参数
2 ./myapp -Xmx64m -Dspring.profiles.active=prod
3
4 # 但某些参数需要在构建时指定
5 native-image -jar app.jar -H:+ReportExceptionStackTraces
```

 Bash

5 多语言编程

GraalVM 的 Polyglot API 允许在 Java 中调用其他语言代码。

5.1 支持的語言

- JavaScript (Node.js 兼容)
- Python
- Ruby
- R
- LLVM (C/C++等)
- WebAssembly

5.2 Java 调用 JavaScript

```
1 import org.graalvm.polyglot.*;
2
3 public class PolyglotExample {
4     public static void main(String[] args) {
5         try (Context context = Context.create()) {
6             // 执行JavaScript代码
7             Value result = context.eval("js", "1 + 2");
8             System.out.println(result.asInt()); // 输出: 3
9
10            // 调用JavaScript函数
11            context.eval("js", "function add(a, b) { return a + b; }");
12            Value addFunction = context.getBindings("js").getMember("add");
13            int sum = addFunction.execute(10, 20).asInt();
14            System.out.println(sum); // 输出: 30
```

 Java

```
15
16     // 访问JavaScript对象
17     context.eval("js", "var person = {name: 'Alice', age: 30};");
18     Value person = context.getBindings("js").getMember("person");
19     System.out.println(person.getMember("name").asString()); // Alice
20 }
21 }
22 }
```

5.3 Java 调用 Python

```
1  import org.graalvm.polyglot.*;
2
3  public class PythonExample {
4      public static void main(String[] args) {
5          try (Context context = Context.newBuilder()
6              .allowAllAccess(true)
7              .build()) {
8
9              // 执行Python代码
10             context.eval("python", "print('Hello from Python!')");
11
12             // 调用Python函数
13             context.eval("python", """
14                 def fibonacci(n):
15                     if n <= 1:
16                         return n
17                     return fibonacci(n-1) + fibonacci(n-2)
18             """);
19
20             Value fibFunc = context.getBindings("python").getMember("fibonacci");
21             int result = fibFunc.execute(10).asInt();
22             System.out.println("Fibonacci(10) = " + result); // 55
23         }
24     }
25 }
```

5.4 数据共享

不同语言之间可以共享数据：

```
1  import org.graalvm.polyglot.*;
2
3  public class DataSharing {
4      public static void main(String[] args) {
5          try (Context context = Context.create()) {
6              // 创建Java对象
7              java.util.Map<String, Object> map = new java.util.HashMap<>();
8              map.put("name", "Alice");
9              map.put("age", 30);
10
```



```
11         // 传递给JavaScript
12         context.getBindings("js").putMember("data", map);
13
14         // 在JavaScript中访问
15         Value result = context.eval("js", "data.name + ' is ' + data.age + ' years old'");
16         System.out.println(result.asString()); // Alice is 30 years old
17     }
18 }
19 }
```

NOTE

多语言编程适合混合语言架构的场景，如使用 Python 进行数据分析，使用 Java 处理业务逻辑。但在生产环境中需要谨慎评估性能和复杂度。

6 性能调优

6.1 构建优化

减少镜像大小

```
1 # 移除未使用的代码
2 native-image -jar app.jar --no-fallback
3
4 # 压缩镜像
5 native-image -jar app.jar -Ob
6
7 # 启用优化
8 native-image -jar app.jar -O2
```

 Bash

并行编译

```
1 # 使用多个线程编译
2 native-image -jar app.jar -J-Xmx4g -J-XX:ParallelGCThreads=4
```

 Bash

6.2 运行时优化

内存管理

```
1 # 设置最大堆大小
2 ./myapp -Xmx128m
3
4 # 使用G1 GC (如果支持)
5 ./myapp -XX:+UseG1GC
```

 Bash

线程优化

```
1 # 设置线程栈大小
2 ./myapp -Xss256k
3
```

 Bash

```
4 # 限制线程数
5 ./myapp -Djava.util.concurrent.ForkJoinPool.common.parallelism=4
```

6.3 监控与调试

启用追踪

```
1 # 报告异常堆栈
2 native-image -jar app.jar -H:+ReportExceptionStackTraces
3
4 # 生成构建报告
5 native-image -jar app.jar -H:GenerateDebugInfo=1
```

 Bash

性能分析

```
1 # 使用perf分析 (Linux)
2 perf record ./myapp
3 perf report
4
5 # 使用async-profiler
6 ./profiler.sh start <pid>
7 ./profiler.sh stop <pid>
```

 Bash

TIP

使用 `-H:+PrintAnalysisCallTree` 可以在构建时查看调用树，帮助识别性能瓶颈。

7 实际应用场景

7.1 Serverless 函数

GraalVM 原生镜像非常适合 Serverless 场景：

优势：

- 冷启动时间短（毫秒级）
- 内存占用低（降低成本）
- 按需扩展

示例：AWS Lambda + GraalVM

```
1 public class LambdaHandler implements RequestHandler<Map<String, Object>, String> {
2
3     @Override
4     public String handleRequest(Map<String, Object> input, Context context) {
5         return "Hello from GraalVM Lambda!";
6     }
7 }
```

 Java

效果：

- 冷启动：从几秒降低到几百毫秒

- 成本：降低 50-70%

7.2 微服务

在微服务架构中，GraalVM 可以显著提升整体性能：

优势：

- 快速启动（适合频繁重启）
- 低内存占用（更多实例）
- 快速扩缩容

示例：Kubernetes 部署

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: graalvm-service
5  spec:
6    replicas: 10
7    template:
8      spec:
9        containers:
10       - name: app
11         image: myapp:latest
12         resources:
13           requests:
14             memory: "32Mi" # 只需32MB
15             cpu: "100m"
16           limits:
17             memory: "64Mi"
18             cpu: "200m"
```

效果：

- 相同硬件可运行更多实例
- 自动扩缩容响应更快

7.3 CLI 工具

GraalVM 非常适合构建命令行工具：

优势：

- 单文件分发
- 无需安装 JVM
- 启动速度快

示例：

```
1  @Command(name = "mytool", description = "My CLI Tool")
2  public class MyTool implements Callable<Integer> {
3
4    @Option(names = {"-n", "--name"}, description = "Your name")
5    String name;
```

```
6
7     @Override
8     public Integer call() throws Exception {
9         System.out.println("Hello, " + name + "!");
10        return 0;
11    }
12
13    public static void main(String[] args) {
14        int exitCode = new CommandLine(new MyTool()).execute(args);
15        System.exit(exitCode);
16    }
17 }
```

```
1 # 构建
2 native-image -jar mytool.jar
3
4 # 分发单个文件
5 ./mytool --name Alice
```

 Bash

7.4 边缘计算

在资源受限的边缘设备上：

优势：

- 小内存占用
- 快速启动
- 无需 JVM 环境

应用场景：

- IoT 设备
- 嵌入式系统
- 移动边缘计算

NOTE

GraalVM 在边缘计算场景中可以显著降低硬件要求，使得在低端设备上运行 Java 应用成为可能。

8 局限性与注意事项

8.1 主要限制

反射限制

- 需要在构建时配置
- 动态类加载不支持
- 增加了配置复杂度

动态特性限制

- 不支持 `Class.forName()` 动态加载

- JNI 需要特殊配置
- 动态代理需要配置

构建时间长

1 传统JAR构建：10-30秒	text
2 Native Image构建：2-10分钟	

影响：CI/CD 流水线变慢

平台相关性

- 需要为目标平台构建
- 跨平台编译复杂
- 需要安装对应平台的工具链

8.2 不适合的场景

场景	原因	建议
长时间运行服务	JIT 优化更好	使用传统 JVM
高度动态的应用	反射配置复杂	评估成本
快速迭代的开发	构建时间长	开发时用 JVM
需要热部署	不支持	使用传统 JVM

8.3 最佳实践

开发阶段

- 使用传统 JVM 进行开发和测试
- 定期构建 Native Image 验证兼容性
- 使用 Spring Boot DevTools 加速开发

测试阶段

- 编写集成测试验证 Native Image
- 测试所有反射和资源访问路径
- 性能基准测试

生产阶段

- 监控内存和 CPU 使用
- 准备回滚方案
- 逐步灰度发布

TIP

推荐策略：开发和测试使用传统 JVM，生产环境根据实际需求选择 JVM 或 Native Image。

9 生态系统与支持

9.1 框架支持

| Spring 生态

- Spring Boot 3.0+: 原生支持
- Spring Framework 6.0+: AOT 处理
- Spring Cloud: 部分支持

| Micronaut

- 天生为 Native Image 设计
- 编译时依赖注入
- 优秀的 GraalVM 支持

| Quarkus

- “Supersonic Subatomic Java”
- 专为 Kubernetes 和 GraalVM 优化
- 最快的启动速度

框架	启动速度	内存占用	学习曲线	生态
Spring Boot	快	低	低	丰富
Micronaut	很快	很低	中	良好
Quarkus	最快	最低	中	增长中

9.2 云服务支持

- AWS Lambda: 支持 Custom Runtime
- Google Cloud Run: 完美支持
- Azure Functions: 支持
- Alibaba Cloud FC: 支持

9.3 社区资源

- 官方文档: <https://www.graalvm.org/>
- GitHub: <https://github.com/oracle/graal>
- Spring Boot 指南: <https://spring.io/guides/topicals/spring-boot-graalvm>
- Awesome GraalVM: <https://github.com/akullpp/awesome-graalvm>

本章完 aalVM